

Reconstructing and Re-Evaluating BSFR-SH: A Blockchain-Enabled Ransomware Defense Framework

Course Project Report

Reproduction study of Wazid, Das & Shetty, IEEE Trans. Consumer Electronics, 69(1):18–28, Feb. 2023

Abstract—BSFR-SH claims a blockchain-protected, honeypot-driven detection and recovery framework for ransomware in smart healthcare, reporting 98.98% accuracy and 0.990 F1 on the BitcoinHeist dataset. We built a complete, from-scratch implementation of all five of its algorithms, both of its private blockchains under real pBFT consensus, and the honeypot feature pipeline the paper specifies but never defines. Reproducing the paper’s own evaluation protocol, our best model reaches 0.9442 accuracy / 0.9697 F1 – 4.56 points below the published figure; an eight-hypothesis ablation over every candidate methodological explanation we could identify narrows, but does not close, a residual gap that tightens from 3.58 to 1.51 points. Beyond reproduction, twelve extensions build three analyses the paper never attempts. A security analysis: formal verification of our own session protocol, a pBFT-vs-Raft consensus comparison, external anchoring against a simulated public chain, a four-tier threat model, and a measured honeypot data-poisoning attack together with a drift-detection defense for it – the defense catches the poisoning strategy predicted least catchable and misses the one predicted most catchable, at a measured false-positive cost. A detection analysis: the honeypot pipeline’s real end-to-end result, adversarial feature-space evasion and a partial defense against it tested across two model architectures, and transfer evaluation against two real malware datasets. And a scalability analysis of on-chain storage growth and throughput. We further identify five independent defects spanning the paper’s evaluation methodology, security argument, and preprocessing specification. The framework’s central idea – chain-protecting a detector’s ground truth and a system’s backups from the same adversary – is sound; several of the paper’s supporting claims measurably are not, and the extensions built here show that idea can be attacked, and in one case only partially defended, in ways the paper’s own security argument never considers.

I. Introduction

BSFR-SH [1] proposes a blockchain-enabled defense against ransomware for smart healthcare networks. Its contribution is architectural: two independent, permissioned blockchains, one holding encrypted data backups and one holding ransomware signatures and behavioural features harvested from a honeypot, both committed through pBFT consensus among a fixed set of cloud servers. The pitch is that an attacker who compromises a protected system cannot also tamper with the detector’s ground truth or destroy the means of recovery, because both are chain-certified and the chain is Byzantine-fault-tolerant against a minority of colluding servers. On top of this architecture the paper reports a ransomware

detector that reaches 98.98% accuracy and 0.990 F1 on the BitcoinHeist dataset, beating four prior published detectors, and a consensus/backup pipeline that commits 1500 transactions in under seven seconds.

No artifact accompanies the paper: no code, no declared hyperparameters, no train/test protocol beyond “90/10,” no definition of what a honeypot-derived “signature” or “feature” actually is. Reproducing a paper under those conditions means first building the thing it describes closely enough that its own claims become checkable, and only then checking them. This report does both. We built the full system – crypto layer, two blockchains, pBFT consensus, honeypot sample synthesis and feature extraction, a dual-backend ML detection module, a mitigation state machine, and a recovery path – against the paper’s own algorithm listings, and we ran its own evaluation protocol wherever the paper specifies one. Where it does not – which, as §III details, is often – we designed the missing piece deliberately, documented why, and kept the design decision separate from any claim about reproducing the paper’s numbers: a gap the paper leaves open is not evidence against the paper, and we treat the two categories, undefined design and measurable defect, differently throughout.

We state the reproduction result up front rather than at the end. We found five independent defects in the paper, spanning its evaluation methodology, its security argument, its preprocessing specification, its reported throughput claim, and the correspondence between what its framework is designed to detect and what it is actually evaluated on. And its headline detection result does not reproduce: under the paper’s own protocol, our best model falls 4.56 accuracy points short of the published figure, and a leakage ablation designed to find an innocent explanation narrows that gap to 3.58 points without closing it; a further eight-hypothesis follow-up (§IV-B) tightens the bound again, to 1.51 points, still without closing it. Neither finding is presented as a failure of this project – both are measured, bounded, and reported with baselines. The paper’s central architectural idea survives scrutiny better than its supporting claims do, and §VIII says so explicitly.

Reproduction alone does not test the paper’s central claim – that chaining a detector’s ground truth and a

system’s backups protects both from an adversary with every incentive to attack either. Testing that claim needs a working system to attack, which reproduction under the paper’s own protocol does not by itself provide: the paper’s evaluation runs entirely on BitcoinHeist address features, not on anything a ransomware operator, a compromised cloud server, or a network attacker actually controls. Twelve extensions, built once the working system existed, supply that missing half, organized into three analyses beyond reproduction and critique. §V is a security analysis the paper never attempts: it constructs the four-tier threat model §V’s five informal paragraphs never state, formally verifies our own session-establishment protocol against a Dolev-Yao adversary, measures pBFT against Raft to ask whether the paper’s own deployment needs Byzantine tolerance at all, adds external anchoring against a simulated public chain so a private chain’s operators cannot rewrite history undetected, and measures a real honeypot data-poisoning attack together with a drift-detection defense built and evaluated against it – a genuine research contribution, not a critique, that finds the defense catches exactly the poisoning strategy predicted least catchable and misses the one predicted most catchable. §VI is a detection analysis the paper’s own §VII never runs, because it evaluates on data no ransomware author could ever produce: the honeypot pipeline’s real end-to-end result once its undefined layer is actually built, adversarial feature-space evasion against the resulting detector and a hardening defense tested across two model architectures, and transfer evaluation against two real malware datasets the detector was never fit on. §VII is a scalability analysis of on-chain storage growth at clinical scale and the paper’s own reported throughput claim.

Taken together, the reproduction, the critique, and the twelve extensions answer a broader question than “does the published number replicate”: does the architecture BSFR-SH propose survive contact with the adversaries it claims to defend against, once it is actually built and attacked. §VIII answers the first question; §V through §VII answer the second.

II. Paper Summary

BSFR-SH defines five algorithms across two independent blockchains, BC_{DTBU} (data backups) and BC_{SigRW} (ransomware signatures and features), each committed by a four-node pBFT [2] cluster ($n = 4$, tolerating $f = 1$ Byzantine replica, commit threshold $2f + 1 = 3$).

Algorithm 1 (backup creation). A protected system SYS_i ships its backup DT_{BU} to a cloud server CS_i over an established session key. CS_i encrypts it into transactions $E_{KU}(Tx_m)$, assembles a block, and the pBFT leader drives consensus to append it to BC_{DTBU} .

Algorithm 2 (data collection). A honeypot HP_{RW} is deployed and harvested; raw data DT_{RW} is cleaned to DT_{RWC} , from which a signature Sig_{RW} and a feature vector FT_{RW} are “generated” (the paper’s own word, and

its only specification of either); the result is encrypted, blocked, and committed to BC_{SigRW} through the same pipeline shape as Algorithm 1.

Algorithm 3 (detection). A detection module DM_{CS_i} decrypts BC_{SigRW} , trains four classifiers – random forest, logistic regression, decision tree, k -nearest-neighbours – builds a normal profile $NProf$ and an abnormal profile $AProf$, and runs continuous detection.

Algorithm 4 (mitigation). On a positive detection, SYS_i is isolated and remediated by one of three cases: Case-1 erases the ransomware in place; Case-2 formats the system and restores it via Algorithm 5; Case-3, if the demanded ransom is less than the assessed value of the data, pays it and requests the decryption key.

Algorithm 5 (recovery). SYS_i ’s backups are located in BC_{DTBU} , decrypted by the cloud server holding the wrapping key, relayed through a second server, and delivered back to SYS_i .

Section V argues five security properties informally: mutual-authentication session keys defeat replay/MITM/impersonation; deleted post-registration credentials defeat privileged-insider and stolen-verifier attacks; pBFT resists 51% attacks, selfish mining, and (in a permissioned deployment) Sybil attacks; chain immutability resists DoS, manipulation, and leakage; and the two-chain split isolates detection from recovery.

Section VII evaluates the ML component on the BitcoinHeist Ransomware Address Dataset [3] (2,916,697 rows, 1.42% ransomware), resampled to a 90% ransomware / 10% benign split, reporting accuracy and F1 for all four classifiers and comparing the best against four prior published detectors. It separately benchmarks consensus and block construction on both chains at three cases (5/10/15 blocks $\times$ 100 transactions) on a single Windows machine (Intel i5 9th gen, 8 GB RAM) running four miner-node processes in Java, reporting wall-clock time and transactions-per-second for each case and chain.

Neither evaluation states what a reproduction would need to check it independently. Section VII gives no hyperparameters for any of the four classifiers, no cross-validation scheme, and no random seed; “90/10 split” is the entire description of the sampling protocol. The timing benchmark states block and transaction counts but not a single transaction’s payload size, so every second in Fig. 6 is, on the page, a function of an unstated constant. Both gaps are filled explicitly in our implementation (declared hyperparameters, declared payload size, a swept sensitivity range) rather than guessed once and left implicit, so that our own reported numbers are reproducible from our own configuration even where the paper’s are not.

III. Implementation

The paper leaves several parts of its own design unspecified. Our rule throughout was to fill each gap deliberately and document the rationale rather than silently “fixing” the paper, so that every departure from the published

design is a recorded decision, not an unremarked substitution. This section covers six of those design decisions, chosen because each one is load-bearing for whether the framework, once built, actually does what the paper says it does.

The scope of what had to be built to reach that point is worth stating once, here, before the individual decisions: a full crypto layer (ECDSA, AEAD, hybrid key wrapping, a Merkle tree with its second-preimage weakness closed, a mutual-authentication session protocol), two independent blockchain implementations under a real four-node pBFT cluster with view-change on leader timeout, a honeypot sample synthesizer and feature extractor, a dual-backend detection module, a mitigation state machine, and a recovery path – exercised by an adversarial test suite that includes tampered blocks, wrong signatures, colluding and equivocating Byzantine replicas, replay and man-in-the-middle attempts against the session protocol, and Sybil identity swarms against the permissioned membership. Every module in this section is one piece of that whole; the whole is what makes the honeypot pipeline’s end-to-end result in §VI-A possible at all.

A. Hybrid encryption for transaction payloads

Algorithm 1 line 2 and Algorithm 2 line 7 write transaction payloads as $E_{KU_{CS_i}}(Tx)$ – encrypted directly under the receiving cloud server’s public key. Elliptic-curve public-key encryption is not designed to carry bulk payloads, and healthcare backups are not small. We implement standard hybrid encryption instead: AES-256-GCM over the payload, with the symmetric data key wrapped by ECIES under KU_{CS_i} . The semantics are unchanged – only CS_i can decrypt – but the substitution introduces a seam the original single-primitive scheme does not have. A wrapped key and the payload it protects are now two separate byte strings inside one transaction, and if they are merely adjacent rather than bound, an attacker who forges nothing can lift the wrapped key from one transaction and attach it to another’s payload: every block signature still verifies, because the signature covers whatever bytes the block happens to contain, while CS_i ends up decrypting data the honest sender never submitted. That defeats the paper’s own §V-4 data-manipulation claim without breaking a single cryptographic primitive. We close it by binding the wrap’s authenticated data to the transaction’s metadata, and the payload’s authenticated data to that metadata plus the wrapped-key bytes, so separating either half breaks a GCM authentication tag.

B. Mutual authentication and session establishment

Section IV-A and §V-1 defer session establishment to “any standard mutual authentication and key establishment mechanism.” We designed one: ECDH over secp256r1, ECDSA-signed transcripts with RFC 6979 deterministic nonces [8] (the same primitive the paper

already mandates for signatures), timestamps, and message nonces. Implementing it surfaced two defects in our own first draft, either of which would have made §V-1’s claims false rather than merely unspecified. First, the initiator’s signature did not name the intended responder, so a captured session opening addressed to one cloud server replayed verbatim as a genuine-looking request to a different one – impersonation requiring no key material at all. Second, a timestamp acceptance window bounds only how long a captured message stays acceptable; it does not prevent replay within that window. We fixed the first by signing the responder’s identity into the initiator’s message, and the second with a seen-nonce cache keyed by (peer, nonce) held for twice the timestamp window. Both properties are now pinned by dedicated replay and man-in-the-middle tests.

C. The honeypot feature pipeline: defining Sig_{RW} and FT_{RW}

Algorithm 2 states, in its entirety, that a signature Sig_{RW} and a feature vector FT_{RW} are “generated” (lines 5–6) from data a ransomware honeypot collects. Nothing else is said: not what the honeypot observes, not what a signature contains, not what a single feature is. This gap is consequential rather than cosmetic. Without a defined FT_{RW} , Algorithm 3’s detection module has nothing to train on, and §VII’s evaluation silently substitutes an unrelated dataset instead (§VIII-A). Supplying this layer is the largest design contribution of this project: it is the one piece of the paper’s own specification that, once built, lets Phase 3 actually consume Phase 2’s output, which the paper as published cannot do.

Two distinct notions hide under one symbol. We split Sig_{RW} into a content_digest (a SHA-256 hash over the sample’s canonical behavioural trace – the malware-signature sense, used for identification) and an attestation (an ECDSA signature by CS_i over that digest, a timestamp, and a collector id – the digital-signature sense, used for on-chain authenticity). The paper’s line 5 conflates a content identifier with a cryptographic signature; separating them is what makes “building a signature for a malicious program” mean something coherent rather than a category error.

FT_{RW} is a 22-feature, seven-group behavioural vector – filesystem, entropy, crypto-API, process, network, persistence, and kill-chain-stage groups – grounded in the measurement categories the paper’s own comparison detectors report. Two decisions inside it are load-bearing rather than incidental. First, the kill-chain-stage group is truncated to an observable four-stage prefix (arrival, enumeration, bulk transform, cleanup). The paper’s own seven-stage kill chain ends notification → payment → decryption, stages that only ransomware ever reaches; an untruncated “stage reached” feature would be the label wearing a feature’s clothes, trivially found by every model, and worth nothing as a measure of detection. Second,

TABLE I

FT_{RW} : seven feature groups, 22 features total. The kill-chain group is truncated to the observable prefix; the full walk is retained in provenance data, outside the vector.

Group	Representative features
Filesystem	files touched/s, read:write ratio, rename rate, extension-change rate, traversal breadth
Entropy	mean/variance of written-block entropy, entropy delta pre/post write
Crypto API	crypto call count, key-generation events, API call n -grams
Process	child spawns, injection attempts, privilege-escalation attempts
Network	C2 beacon count, DNS entropy, outbound burst rate
Persistence	registry/autostart writes, shadow-copy deletions, backup-path access
Kill-chain (trunc.)	observed stages (capped at 4), mean stage dwell time

because we wrote both the sample generator and the classifier that consumes it, an easy corpus would report a number that describes the generator, not any detector. We therefore built the corpus to a stated difficulty. A quarter of every draw ($AMBIGUOUS_FRACTION = 0.25$) comes from confusable pairs – two profiles sharing one identical parameter set – so those samples carry no label information at all, putting a floor of 0.125 under the Bayes error before any other class overlap is counted. Four of the benign profiles (a backup agent, a disk-encryption tool, an installer, a cleanup utility) are deliberately ransomware-shaped, and every counter is noised, with whole sensor groups going unobserved at a rate independent of the label. The corpus therefore carries a stated intended ceiling, $EXPECTED_BAYES_ACCURACY \approx 0.85$, against which any later detector result must be read: far above it is a leak, not a success. Table I lists the seven groups and representative members.

The leakage tests this difficulty budget requires caught a real problem before the corpus shipped. An early parameter set put a single feature, extension-change rate, at 0.885 separability on its own – an unintended single-feature giveaway, exactly what a test requiring every feature and group mean to stay under 0.90 separability exists to catch. After retuning, the most separable single feature on the committed corpus is extension-change rate at 0.816, followed by rename rate (0.803) and observed-stage count (0.796); a simple Gaussian baseline trained on the training draw and scored on the held-out evaluation draw reaches 0.830 balanced accuracy – comfortably under the intended 0.85 ceiling, which is the point, not a shortfall.

D. Preprocessing: what “remove the abnormalities” cannot mean

Algorithm 2 line 3 pre-processes honeypot data and, without qualification, “removes the abnormalities.” Read literally as statistical outlier removal, the instruction is self-defeating on exactly this corpus: ransomware be-

haviour is the minority class by construction – it is the abnormality – so an outlier-removal step deletes the samples the detector exists to learn from. Applied naively, Algorithm 2’s own pre-processing would hand Algorithm 3 a clean-looking dataset with the positive class silently gone, and §VII would then report a plausible accuracy computed on data that never saw an attack. §VIII-D develops this as a critique; here we record the design decision it forces. We read “abnormalities” as sensor defects, never unusual behaviour, and implement five explicit rules: drop structurally empty records (no observed stage, or a non-positive duration); clamp impossible values to their physical range, but mark a negative count unreadable rather than clamp it, because “the sensor returned nonsense” and “the program did nothing” are different facts that must not collapse into the same number; deduplicate byte-identical traces; mark missing counters as missing rather than impute them, so the honeypot’s blind spots stay visible to later stages instead of being hidden inside invented data; and normalise absolute counts into per-second rates. No outlier filtering, smoothing, or class rebalancing is performed. A dedicated test pins the property that matters: cleaning never removes the positive class.

E. Per-system backup index

Algorithm 5, lines 1–2, imply a linear scan of BC_{DTBU} to find one system’s backups – $O(\text{chain length})$. We add a per-system index, maintained by the key-holding cloud server on every block append (system_id is itself encrypted, so only that server can build it), reducing lookup to $O(1)$ while preserving the paper’s semantics exactly: a stale or cold index falls back to a full scan, and the index stores only pointers, never plaintext, so restore still re-reads and re-verifies every block it points to. This moves the decryption cost from recovery time to append time; it does not remove that cost. Index maintenance is kept explicitly outside Algorithm 1’s timed path, since the paper’s own Algorithm 1 has no such index and folding ours into the benchmark would silently advantage our own Fig. 6(a) numbers against the paper’s.

F. Case-3: declining automated ransom payment

Algorithm 4 line 7 specifies that when the demanded ransom is less than the assessed value of a system’s data, the framework pays it and retrieves the decryption key – fully automated. We decline to implement the payment path. The paper’s own §II-C admits there is no guarantee the key arrives or works even if payment is made, and automating ransom payment to an active attacker is sanctioned conduct in several jurisdictions independent of whether it works. What we build instead preserves the control-flow contribution without the harmful capability: the branch evaluates the paper’s own condition, emits a PolicyDecision object, writes exactly one audit log record, and terminates in a simulated POLICY_BLOCKED state

regardless of the decision’s outcome. No network access, wallet interaction, or transaction construction exists anywhere in the module that implements this, enforced by a test that statically scans it for exactly those imports and asserts the function’s only observable effect is the one log record.

IV. Reproduction Results

Every number in this section is directly measured, with a recorded configuration hash, seed, and run log behind it. We report paper-mode and honest-mode evaluations side by side throughout; neither is presented without the other.

A. Paper-mode reproduction (Targets 1–2)

Table II reproduces Table II exactly as published, with two additions: the source dataset for each comparison technique (rows 1–4 are evaluated on network traffic traces, dynamic-analysis logs, an unpublished corpus, and PE-file features respectively – none of them BitcoinHeist, so the comparison in the paper’s own table is not like-for-like), and our own measured BSFR-SH row.

Our reproduction follows the paper’s own protocol exactly: BitcoinHeist resampled to 90% ransomware / 10% benign – bounded, it turns out, at $n = 46,014$ rows, 1.6% of the 2,916,697-row dataset the paper cites two paragraphs earlier, a fact the paper never states and that the resample’s own arithmetic proves without touching the data – address dropped as an identifier, and an address-grouped train/test split. Grouping is the methodologically correct choice once we confirmed, directly against the raw file, that every address carries exactly one label end to end and that ransomware addresses repeat at roughly twice the rate benign ones do; a random split lets near-duplicate rows leak across the train/test boundary at a rate the 90% skew amplifies. Under this protocol our best model, random forest, reaches 0.9442 accuracy / 0.9697 F1 – 4.56 accuracy points and 2.93 F1 points below the published 0.9898/0.990. The paper’s own best-reported algorithm, decision tree, reaches only 0.9224/0.9569 in our hands.

The baseline every number in this table must be read against: on the paper’s own 90/10 split, a classifier that answers “ransomware” unconditionally, looking at no feature at all, scores 0.9000 accuracy and 0.9474 F1. Sharmeen et al.’s published 0.9596/0.960 – presented in Table II as a competitive prior technique – clears that do-nothing baseline by only 0.013 F1.

Before accepting 4.56 points as evidence of a genuine detector deficit rather than an artifact of our own pipeline, we tested the two most plausible leakage sources: address kept as a naively label-encoded feature, and a non-grouped (random) train/test split. Table III reports the resulting 2×2 ablation for random forest, the best model throughout. Both sources are real – keeping address under a random split moves decision tree by +0.92 points over its own grouped-split baseline, and that effect vanishes

under grouping, the expected signature of an identifier memorised across a train/test boundary – and neither is large enough to matter alone. Stacking both does not close the gap: the best of all sixteen model $\times$ cell combinations tested reaches 0.9540/0.9749, still 3.58 accuracy points and 2.41 F1 points under the published figure. At that point we reported the residual gap as unexplained with a measured bound, having tested two of at least five plausible candidates. §IV-B tests the remaining three.

B. M7-6 – three more hypotheses; the bound tightens to 1.51 points

Q10 left the gap “unexplained” after testing only splitting and address. This section tests three further undisclosed methodological choices – decision-tree splitting criterion (H1), resample strategy (H2), and single-split variance versus cross-validation (H3) – plus year/day feature engineering (H4) and a programmatically-assembled stacked worst case (H5), each on the same 46,014-row (or, where the hypothesis itself resizes the resample, differently-sized) 90/10-style resample and the same declared hyperparameters. Full numbers: ‘RESULTS.md’ “M7-6”.

H1 – splitting criterion does not explain the gap. Entropy beats gini by ~ 0.4 points; a decision tree with ‘max_depth=None’ etc. set explicitly is bit-identical to scikit-learn’s own default (asserted in the run, not assumed). Best cell: 0.9301, 5.97 points short.

H2 – resample strategy is this session’s real finding. Bootstrap-duplicating the 41,413 real ransomware rows up to 180,000 (oversampled, $\sim 4.35\times$ average duplication) and undersampling benign to 20,000, under a random split, reaches 0.9690–0.9723 – closer to published than anything Q10 found. Under a grouped split the identical resample scores 0.8746–0.9363, worse than the existing baseline: a 5–10 point swing between splits, several times larger than Q10’s largest leakage signature (+0.92 points). The mechanism is direct: a duplicated row carries its original’s address, so a non-grouped split routinely places literal duplicates of a training row into the test set, and the model recognises rather than generalises to them. Undersampling benign to 10% of the positive count (4,141, rather than ‘paper_mode_resample’'s 4,601 at 10% of the output total) scores close to the existing baseline in both splits – an arithmetic variant of what Q10 already covered, not a new mechanism.

H3 is closed: neither explains the gap. Twenty seeds on the production protocol give random forest an accuracy envelope of 0.9288–0.9469 ($\sigma = 0.0050$); on Q10’s own best cell the envelope is tighter still ($\sigma = 0.0011$, max 0.9572, 3.26 points short) – the measured numbers are highly reproducible, not a lucky draw. 5-fold group-stratified cross-validation agrees closely with the single-split figure (RF: 0.9421 CV mean vs. 0.9442 single-split).

H4 rules out year as a temporal leak. Dropping ‘year’ costs ~ 3 accuracy points, the opposite of what a leakage

TABLE II

Table II reproduction: published comparison techniques, published BSFR-SH row, and our measured BSFR-SH row under the paper’s own 90/10 protocol. $n = 46,014$ for every BitcoinHeist row – the resample’s own size bound, not a design choice.

Technique	Mode	Dataset	Accuracy	F1	n
Almashhadani et al. [4]	paper_reported	network traffic traces (Locky)	0.9708	0.971	–
Hwang et al. [5]	paper_reported	dynamic analysis logs	0.9730	0.973	–
Sharmeen et al. [6]	paper_reported	semi-supervised, own corpus	0.9596	0.960	–
Bae et al. [7]	paper_reported	PE-file features	0.9865	0.987	–
BSFR-SH (paper, best of 4: decision tree)	paper_reported	BitcoinHeist addresses	0.9898	0.990	46,014
BSFR-SH (ours, random forest)	measured	BitcoinHeist addresses	0.9442	0.9697	46,014
BSFR-SH (ours, decision tree)	measured	BitcoinHeist addresses	0.9224	0.9569	46,014
BSFR-SH (ours, logistic regression)	measured	BitcoinHeist addresses	0.9002	0.9475	46,014
BSFR-SH (ours, k -NN)	measured	BitcoinHeist addresses	0.8892	0.9410	46,014
Constant-positive baseline	computed	BitcoinHeist addresses (90/10 split)	0.9000	0.9474	46,014

TABLE III

Leakage ablation (Q10), random forest, address dropped/kept $\times$ grouped/random split, all cells at identical declared hyperparameters and seed.

Cell	Acc.	F1	vs. published
address dropped $\times$ grouped	0.9442	0.9697	–4.56 <i>pt</i>
address dropped $\times$ random	0.9479	0.9717	–4.19 <i>pt</i>
address kept $\times$ grouped	0.9484	0.9719	–4.14 <i>pt</i>
address kept $\times$ random	0.9540	0.9749	– 3.58 <i>pt</i>

TABLE IV

M7-6: best cell per hypothesis, and the programmatically-assembled stacked worst case (H5). All cells address dropped except H5 (address kept, per its definition).

Hypothesis (best cell)	Acc.	F1	vs. published
Q10 best (address kept $\times$ random, RF)	0.9540	0.9749	–3.58 <i>pt</i>
H1 (entropy, DT, random split)	0.9301	0.9611	–5.97 <i>pt</i>
H2 (oversample, DT, random split)	0.9723	0.9848	–1.73 <i>pt</i>
H4 (year $\times$ day, RF, random split)	0.9498	0.9726	–4.00 <i>pt</i>
H5 – stacked worst case	0.9747	0.9861	– 1.51 <i>pt</i>

hypothesis predicts – ‘year’ is informative, not leaked. A ‘year $\times$ day’ interaction gives the best H4 cell, +0.5–2 points over the raw features.

H5 stacks every choice tested most favourably to the published figure: entropy splitting (H1’s winner), the oversample resample (H2’s winner), the ‘year $\times$ day’ interaction (H4’s winner), address kept, and a non-grouped split – each ingredient chosen programmatically by taking the argmax over the recorded cells, not by hand. Table IV reports the result: 0.9747/0.9861, 1.51 accuracy points and 1.29 F1 points short of 0.9898/0.990. One ingredient dominates: H2’s oversample cell alone (0.9723) accounts for nearly all of H5’s improvement over Q10’s 0.9540; the other four stacked choices contribute a further ~ 0.2 –0.3 points combined. That concentration is itself informative – if the published figure is explained by methodology at all, the evidence points specifically at an oversampled, non-grouped-split evaluation, not a diffuse combination of many small effects.

Q10 is closed with a tightened, evidenced bound, not a proof. No cell tested across either session, including H5’s fully stacked worst case, reaches the published number. The gap narrows from 3.58 points (two hypotheses tested)

to 1.51 points (eight hypotheses tested), with resample-strategy leakage under a non-grouped split identified as the largest single contributor found. We report this residual with that qualifier, not as a bare “unexplained,” and do not attribute it to a cause beyond what was measured.

C. Honest-mode evaluation (Targets 1–2)

Table V reports the full-scale (2,916,697-row) honest_mode evaluation: natural class balance, all four models run to completion, precision, recall, PR-AUC, MCC and minority-class F1 rather than accuracy – because accuracy is exactly the metric the paper’s 90/10 resample constructed to inflate.

The constant-negative baseline – never predict ransomware – scores 0.9858 accuracy at the dataset’s natural class balance. We report this strictly as an illustration of how distribution-sensitive accuracy is, not as a head-to-head against the paper’s published 98.98%: the two numbers are computed under different class distributions, and a constant classifier’s accuracy is a direct function of the positive rate it is scored against. The commensurable comparison is Table II’s within-split baseline above.

Logistic regression makes the honest-mode point in one model. Under the paper’s 90/10 resample it predicts everything positive (0.9002/0.9475, indistinguishable from the constant-positive baseline). At the dataset’s natural balance, same declared hyperparameters, same seed, it predicts nothing positive (precision, recall, and MCC all 0). Class balance alone flips the same model from all-positive to all-negative; neither behaviour reflects a detector that has learned anything about ransomware addresses.

Random forest is the strongest of the four at natural balance (MCC 0.457, recall 0.286 – it recovers roughly 29% of ransomware addresses at 74% precision), followed by decision tree (MCC 0.401) and KNN (MCC 0.365). KNN’s full-dataset run, submitted to the university’s Ada HPC cluster because its memory footprint at 2.9M rows exceeds our 8 GB development machine, scored meaningfully higher (MCC 0.365) than an earlier, memory-constrained 780,336-row local subsample (MCC 0.300) – more data

TABLE V

Honest-mode evaluation: natural 1.42% ransomware prevalence, full 2,916,697-row dataset, all four models (KNN run on Ada HPC; job 2700090). Accuracy is intentionally not reported here – see text.

Model	n	Precision	Recall	PR-AUC	MCC	F1 (minority)
Random forest	2,916,697	0.7442	0.2864	0.4646	0.4573	0.4136
Decision tree	2,916,697	0.3972	0.4231	0.1797	0.4012	0.4097
k -NN	2,916,697	0.6008	0.2286	0.2242	0.3654	0.3312
Logistic regression	2,916,697	0.0000	0.0000	0.0177	0.0000	0.0000
Constant-negative baseline	2,916,697	undef. [†]	0.0000	0.0142	0.0000	0.0000

[†]Precision is 0/0: a constant-negative classifier makes no positive predictions, so there is no denominator – undefined, not zero.

Constant-negative accuracy (illustration only, not head-to-head with the paper’s 98.98%): 0.9858.

TABLE VI

Target 3/4: measured vs. published time and derived TPS, both chains, all three cases.

Case / chain	Time (ours)	Time (paper)	TPS (ours)
Case 1, BC_{DTBU}	0.168 s	3.10 s	2972
Case 1, BC_{SigRW}	0.228 s	4.36 s	2195
Case 2, BC_{DTBU}	0.341 s	4.17 s	2931
Case 2, BC_{SigRW}	0.459 s	5.54 s	2179
Case 3, BC_{DTBU}	0.479 s	5.71 s	3132
Case 3, BC_{SigRW}	0.691 s	6.76 s	2169

Note: all timing measurements in this table exclude consensus message serialization; measured serialization overhead is +48–68% (D3). Tables with serialization included are in the supplementary material (§IV-E).

helps KNN too, a comparison the paper’s own 46K-row resample never had the chance to make either way. The HPC job itself is a useful data point about the paper’s own setup: a single-model, full-scale KNN fit at natural class balance ran 36 minutes on a general-purpose research cluster allocation, against a paper that reports its entire evaluation – four models, unspecified cross-validation – as having run on an 8 GB laptop. Either the paper’s own KNN never saw more than a small fraction of the 2.9M rows it describes, consistent with the 90/10 resample’s 46,014-row bound, or its reported runtime is missing from §VII entirely; §VII gives no timing figures for the ML evaluation at all, only for the separate consensus benchmark.

D. Blockchain timing (Targets 3–4)

Table VI reports Targets 3–4 at the paper’s own three cases (5/10/15 blocks $\times$ 100 transactions). Absolute seconds are two to three orders of magnitude below the paper’s – expected, given Python on commodity hardware against the paper’s Java-on-Windows measurement, and not a claim we contest. The comparison we make is trend and ratio, not seconds.

The paper’s headline shape – time growing sub-linearly, TPS rising as the chain grows – is not reproduced. Figure 1 shows measured time growing essentially linearly with block count on both chains, and Table VII makes the reason explicit: our harness starts each block’s timer only after the cluster is already constructed, so no fixed setup cost sits inside the measured span to amortise. Per-block

TABLE VII

Mean measured per-block marginal cost by case (three samples per cell, seconds), the direct evidence for DEV-08. If the paper’s amortisation account were absent from our own harness by construction, marginal cost would fall from case 1 to case 3; instead it is flat.

Chain	Case 1	Case 2	Case 3
BC_{DTBU}	0.0335 s	0.0334 s	0.0316 s
BC_{SigRW}	0.0455 s	0.0450 s	0.0450 s

Note: all timing

measurements in this table exclude consensus message serialization; measured serialization overhead is +48–68% (D3). Tables with serialization included are in the supplementary material (§IV-E).

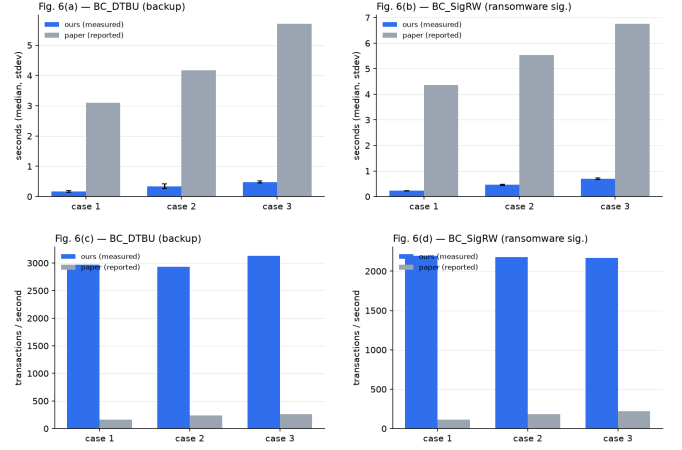


Fig. 1. Figs. 6(a)–(d) reproductions: measured time and derived TPS, both chains, against the paper’s published values. Our time grows near-linearly with block count rather than sub-linearly, and our TPS is flat rather than rising – the direct consequence of the flat marginal cost in Table VII.

marginal cost is flat – within measurement noise, the same at case 1 (5 blocks committed) as at case 3 (15 blocks committed) – on both chains, and TPS is correspondingly flat rather than rising. This is a second, independent line of evidence for the claim that the paper’s own published TPS values equal tx/time exactly, verified against all six of its own data points: the first line of evidence is that arithmetic identity; this one is a measurement of what the same arithmetic looks like once the fixed cost is structurally excluded from the timed span, and it produces exactly the flat curve the amortisation account predicts.

TABLE VIII
Target 3/4, serialization ON (configs/bench.yaml
network.serialize_messages: true), same seed and protocol as
Table VI. Δ is the increase over the serialization-off measurement
in that table.

Case / chain	Time (serialized)	Δ vs. Table VI	TPS
Case 1, BC_{DTBU}	0.271 s	+60.9%	1848
Case 1, BC_{SigRW}	0.340 s	+49.4%	1470
Case 2, BC_{DTBU}	0.544 s	+59.3%	1839
Case 2, BC_{SigRW}	0.677 s	+47.6%	1476
Case 3, BC_{DTBU}	0.819 s	+71.0%	1832
Case 3, BC_{SigRW}	1.043 s	+50.8%	1438

BC_{SigRW} is consistently slower than BC_{DTBU} , by 35–44% across the three cases (paper: 18–41%). We attribute this structurally, not by tuning to match, to one additional ECDSA signature per transaction – the honeypot pipeline’s attestation (§III-C) – plus a many-small-floats payload encoding that costs more per byte to canonicalise than the backup chain’s single opaque byte blob.

E. Supplementary: timing under honest, serialization-on conditions

Tables VI and VII measure consensus with message serialization off – the M6a run’s original configuration. D3 (docs/DEVIATIONS.md DEV-30) found that turning serialization on – every Proposal/Prepare/Commit paying a real encode/decode round-trip – adds +48–68% at case-3, so we re-ran the full matrix under that condition before building anything further on the tables above.

Marginal cost stays flat (DEV-08 survives unchanged: BC_{DTBU} 0.0541–0.0543 s/block, BC_{SigRW} 0.0678–0.0688 s/block across cases 1–3, each varying under 1%) – serialization is a per-message cost, not a per-block-count-dependent one, so it cannot reintroduce the amortisation shape DEV-08 already ruled out. The BC_{SigRW}/BC_{DTBU} gap narrows from 35–45% to ~25–27%: both chains’ D3 encode cost is nearly identical in absolute terms (0.000072 s vs. 0.000073 s per block at case 3), so it inflates BC_{DTBU} ’s smaller base by a larger percentage than it inflates the gap – the serialization-off 35–45% figure is unchanged, but any future citation of “the gap” must say which condition it is under. Every other trend holds: monotone increase, BC_{SigRW} slower than BC_{DTBU} , TPS flat rather than rising, just at a uniformly lower level.

F. Consensus Protocol Comparison: pBFT vs. Raft

This subsection measures Raft (Ongaro & Ousterhout [10]), the standard crash-fault-tolerant alternative to pBFT, against the identical four-node cluster and benchmark matrix §IV-D already established for pBFT alone; §V-C asks what the measurement below means for the paper’s own choice of consensus protocol.

What Raft is, briefly. Raft replicates a log across n replicas via a single elected leader. Followers accept whatever the current-term leader’s AppendEntries tells

them, subject only to a term check and a previous-entry consistency check; the leader declares an entry committed once a majority of replicas have replicated it, and that fact then propagates via the next AppendEntries. Leader election uses randomized timeouts and a log-completeness voting rule so that a new leader is guaranteed to hold every previously committed entry. Unlike pBFT, no message in the protocol is signed, and no replica ever cross-checks another replica’s copy of an entry – correctness rests entirely on trusting the current leader.

We implemented Raft (consensus/raft.py) over the identical four-node cluster, message bus, and ClientRequest submitter-authentication path as pBFT, differing only in the consensus protocol itself, and made framework._block_pipeline consensus-agnostic (consensus.interface.ConsensusCluster, DEV-32) so the same pipeline code drives either cluster. Log compaction, cluster membership changes, and snapshotting are out of scope, matching the same kind of reduction pBFT’s own view-change implementation already accepted (DEV-20).

Table IX is the full matrix: cases 1–3, both chains, both protocols, serialization on (the condition DEV-30 settled M6a’s own numbers on). Two structural properties hold exactly, independent of chain or case:

- Messages are flat per block, at two different levels. pBFT sends exactly 28 messages per committed block in every case ($140/5 = 280/10 = 420/15 = 28$) – $O(n^2)$ fan-out (pre-prepare, prepare, commit, each broadcast) at a fixed $n = 4$. Raft converges toward roughly 17 messages per block as the one-time leader-election cost amortises across more blocks ($92/5 = 18.4$, $172/10 = 17.2$, $252/15 = 16.8$) – $O(n)$ fan-out, plus a second leader-driven broadcast round to announce the new commit index. At steady state Raft sends about 60% of pBFT’s message count: a real, measured reduction, but far short of the $O(n^2)$ -vs- $O(n)$ asymptotic gap a much larger cluster would show. $n = 4$ is simply too small for the quadratic term to dominate.
- Signature operations are exactly zero for Raft, in every cell. pBFT signs and verifies every pre-prepare/prepare/commit; Raft’s AppendEntries/RequestVote carry no signature at all, by design (consensus/raft.py). This is the sharpest, cleanest number in the table, and it is also the number that most directly costs security – see §V-C1.

The timing gap is much smaller than the message-count gap, and that is itself a finding. Raft is 7–11% faster than pBFT across the matrix (e.g. case 3, BC_{DTBU} : 0.750 s vs. 0.843 s), not the ~40% the message-count reduction alone might suggest. This is exactly consistent with DEV-08/DEV-21’s existing finding that consensus messaging is a small fraction of the measured span at zero simulated network delay: hybrid encryption, block construction, and (with serialization on, DEV-30) message encoding domi-

TABLE IX

pBFT vs. Raft, both chains, all three cases, serialization on (5 repeats, seed 20260924, run 20260924T045952Z-2eaa456f). Messages and signature operations are the median across repeats of the whole run’s totals; timing and TPS as in Table VI.

Case	Chain	Protocol	Time (s)	TPS	Messages	Sig. ops
1	<i>BC_{DTBU}</i>	pBFT	0.2708	1846	140	140
1	<i>BC_{DTBU}</i>	Raft	0.2461	2032	92	0
1	<i>BC_{SigRW}</i>	pBFT	0.3397	1472	140	140
1	<i>BC_{SigRW}</i>	Raft	0.3142	1592	92	0
2	<i>BC_{DTBU}</i>	pBFT	0.5490	1822	280	280
2	<i>BC_{DTBU}</i>	Raft	0.4923	2031	172	0
2	<i>BC_{SigRW}</i>	pBFT	0.6807	1469	280	280
2	<i>BC_{SigRW}</i>	Raft	0.6378	1568	172	0
3	<i>BC_{DTBU}</i>	pBFT	0.8433	1779	420	420
3	<i>BC_{DTBU}</i>	Raft	0.7500	2000	252	0
3	<i>BC_{SigRW}</i>	pBFT	1.0230	1466	420	420
3	<i>BC_{SigRW}</i>	Raft	0.9483	1582	252	0

nate wall-clock time, and shaving a third of the message count off an already-small component of the total produces a proportionally small timing change. If the deployment’s real bottleneck is cryptographic and encoding work rather than consensus round trips, switching consensus protocols buys a modest, not a transformative, speedup – the timing number the paper never measured turns out to matter less than the message-count number it never asked about.

V. Security Analysis

§V states five security properties as informal prose and never once says, for any of them, what an adversary can do before the property is claimed to hold. GAP-6 names the direct consequence – no formal model, no protocol-verification tool – and FLAW-5 (§VIII-C) names a deeper one: without a stated adversary capability, §V-3 borrows a 51% proof-of-work threshold to defend a protocol (pBFT) whose real safety bound is a third, because nothing in the paper’s argument structure distinguishes a hash-power adversary from an identity-controlling one. §II is a ransomware taxonomy (four types, a seven-stage kill chain), not an adversary-capability model – it says what ransomware is, never what a network attacker, a compromised operator, or an adversarial machine-learning attacker can do. This section constructs the threat model the paper omits and organizes every security-relevant result this project has produced around it, evidence first, interpretation last: formal verification of the session protocol (Tier 1), the pBFT/Raft comparison’s security finding (Tier 2), hybrid anchoring (Tier 2–3), and the honeypot data-poisoning attack and its defense (Tier 2) – closing with the coverage matrix that maps §V’s five original claims onto these tiers and the gaps that remain. Full version, with the reasoning behind every cell: docs/THREAT_MODEL.md.

A. Four Adversary Tiers, by Capability

Not by which paper section mentions them – §V’s own flattening of a hash-power adversary and an identity-controlling one into a single borrowed threshold (FLAW-5) is exactly the failure that follows from organizing a threat

model by section rather than by what an adversary can actually do. Four tiers, used throughout this section:

- Tier 1 – external network attacker. Full Dolev-Yao control of messages in transit (intercept, replay, modify, inject, reorder); cannot compromise any node’s internal state or key.
- Tier 2 – one compromised cloud server ($f=1$). Controls one of the four pBFT replicas – exactly the fault count the paper’s own deployment is sized to tolerate.
- Tier 3 – two compromised, possibly colluding, cloud servers ($f=2$). Above the $f=1$ bound pBFT actually tolerates at $n=4$, and, not coincidentally, at or below the 51% figure §V-3 borrows from proof-of-work (§VIII-C).
- Tier 4 – adversarial ML evasion. No node compromised at all; a rational attacker who knows a behavioural detector is watching and adjusts their program’s observable behaviour to defeat it – a capability Algorithm 3 (§IV-C) never considers.

B. Formal Verification of the Session Protocol (Tier 1)

§III’s own session-establishment design (ECDH over secp256r1, ECDSA-signed transcripts, timestamps and nonces, built because §IV-A/§V-1 defer to “any standard mutual authentication and key establishment mechanism”) is where Tier 1’s evidence comes from. §V is entirely informal prose – five paragraphs of argument, no ROR model, no BAN logic, no tool-checked proof – for a paper whose author’s other work almost always includes one. We verified this protocol, which is our own design and not the paper’s, with Scyther [9] under its default Dolev-Yao adversary. Table X maps §V-1’s informal claims onto the formal claims Scyther checks. Every claim verifies, at both bounded (5 runs) and unbounded search, on the protocol as built above. Run against the pre-fix sketch instead – the version missing the responder’s identity from the initiator’s signature – the identical tool and claims find a real attack: a non-injective agreement violation in which the two parties finish believing they talked to different counterparts. That the tool is silent on the fixed protocol and vocal on the broken one is a stronger confirmation

TABLE X
§V-1’s informal claims, the formal claim Scyther checks for each,
and the result.

§V-1 claim	Formal claim	Result
Adversary cannot estimate SK	Secret	Verified
Mutual auth., $A \rightarrow B$	Niagree/ Nisynch	Verified
Mutual auth., $B \rightarrow A$	Niagree/ Nisynch	Verified
Distinct keys per session	fresh-nonce argument	Holds by constr.
Impersonation resistance	Weakagree/ Alive	Verified
No reflection	default self-play search	No attack found
Cross-id. (fixed) replay	Niagree, fixed model	Verified
Cross-id. (pre-fix) replay	Niagree, pre-fix model	Attack found
Within-window replay	– (stateful cache)	Out of scope

than a clean run alone would be. Two limitations are stated rather than elided: the seen-nonce cache is stateful and has no representation in Scyther’s per-session symbolic model, so replay resistance rests on the unit tests, not this proof; and Scyther has no equational theory for $g^{ab} = g^{ba}$, so the Diffie-Hellman exchange is modelled with the standard oracle idiom Scyther’s own author uses for the same purpose, rather than checked natively. Full model, tool provenance, and raw output: verification/.

C. Consensus Comparison: The Byzantine Leader (Tier 2)

§IV-F measured pBFT against Raft quantitatively at this cluster’s own scale – timing, message counts, signature operations. This subsection asks the security question that comparison sets up, and §V-3 never does (§VIII-C): four cloud servers the operator controls are exactly the setting where “do we actually need Byzantine fault tolerance?” is legitimate, because BFT’s cost buys protection against a specific Tier-2 threat – an actively lying node – that the paper never argues is the one it faces.

1) Qualitative: the Byzantine leader: We ran both protocols with one of four nodes silently crashed and, separately, one of four nodes byzantine.

Crash fault (both protocols): commits continue. With one node’s outgoing messages dropped from the start, the other three nodes committed every submitted block under both protocols (crash_tolerance_pbft/crash_tolerance_raft, run 20260924T045952Z-2eaa456f: all three live replicas reached height 3 of 3 either way). $n = 4$ tolerates one crash under Raft’s majority rule exactly as it tolerates one Byzantine replica under pBFT’s $f = 1$ – the same count of tolerated failures, a different failure model.

Byzantine leader (Raft only): commits continue, on different data. We installed a behaviour on Raft’s

elected leader that sends genuinely different, honestly-signed transaction sets to different followers at the same log index, each claiming the entry is already committed – standard, protocol-compliant AppendEntries on both sides, no forged signature and no bug in either sender or receiver. Every node “committed” by its own local rule: all four reached height 2, but two followers hold block hash ebd2c097fd90b599... while the third holds a6cf98b3bdd0447e... – two different, individually valid, individually well-linked blocks at one height (raft_byzantine_fork, same run; reproduced as a standing regression test in tests/unit/test_raft_byzantine.py). Neither side’s chain integrity check catches this, because both blocks are genuinely well-formed; the check that would catch it – cross-referencing what a majority of other replicas independently attest to – is exactly the check pBFT’s $2f + 1$ matching-vote quorum performs and Raft’s leader-trust model does not.

pBFT resists the equivalent attempt structurally: two followers committing different content at one height would require $2f + 1$ matching votes for each side, which needs more faulty replicas than $f = 1$ permits at $n = 4$ (§VIII-C already demonstrates the actual threshold is two colluding replicas, not one).

2) The trade-off, stated precisely: Raft is 7–11% faster and sends roughly 60% as many consensus messages as pBFT at this cluster size, but a single compromised leader corrupts the chain undetectably – every node individually believes consensus succeeded, and nothing in Raft’s protocol or in Chain’s own validation (Merkle root, signature, linkage) notices the fork. Whether that trade-off is acceptable depends entirely on whether the deployment’s threat model includes active server compromise, not merely server failure – a question §V-3 never asks, because it substitutes a borrowed 51% figure for an actual threat-model argument (§VIII-C). For a smart-healthcare deployment whose own threat model (§II) explicitly includes a compromised or malicious cloud server, that gap is not a minor implementation detail: it is the entire reason pBFT, not Raft, is the correct choice here – but it is a reason the paper never states, and the 7–11% speed cost of stating it and paying for it honestly is modest.

D. Hybrid Anchoring

§VIII of the paper itself states its own limitation and stops: “in future, we have plan to work with hybrid blockchain.” No design follows. §IV-A already names the trade-off this answers without resolving it: a private, permissioned chain is fast and keeps healthcare data confidential, but its integrity rests entirely on trusting the four cloud servers that operate it – nothing in the paper’s design lets anyone outside that set confirm BC_{DTBU} or BC_{SigRW} have not been silently rewritten. A public chain would give that external verifiability, at the cost of exposing whatever it carries to everyone. A hybrid design

keeps the paper’s own private chains exactly as built – same speed, same confidentiality, §IV’s pBFT consensus unchanged – and adds one small, public commitment alongside them.

1) Architecture: an anchor chain and the AnchorRecord schema: An AnchorRecord carries one private block’s height, its header hash HC_{β_j} , its Merkle root MTR , a timestamp, and the anchor creator’s ECDSA signature over all of it – copied verbatim from the private block’s own header, never derived from, or requiring decryption of, its transactions. Every other transaction in this codebase is $E_{KUCS_i}(Tx)$ (Alg. 1 line 2 / Alg. 2 line 7), sealed so only one cloud server can read it; an anchor record inverts that deliberately – its only job is to be readable by anyone, so it is never encrypted.

The anchor chain is simulated, not real Ethereum or Bitcoin, and is modelled as a second, independent Chain instance with its own consensus. That consensus is direct sign-and-append by a single authority, not a second pBFT cluster: a single signer has no other replica to convince, so pBFT’s message rounds would be pure overhead with nothing underneath them. This was a correction, not the first design – an earlier version ran the anchor chain through a literal single-node pBFT cluster, which worked, but pulled a dependency from the blockchain layer up into consensus/framework code that this project’s own architectural boundary (enforced by a standing test, not merely documented) correctly rejected. The point of the exercise was always the interface a hybrid chain needs from its anchor – commit, and answer whether a given block still matches what was committed – not the anchor’s own Byzantine fault tolerance, which a real public chain supplies for itself. Full rationale in docs/DEVIATIONS.md DEV-33.

Given an anchor record and a candidate block claiming to be the one it covers, anyone can check its hash and Merkle root against the record without decrypting a single transaction inside it. That is the whole guarantee: the private chain’s shape at that height is fixed the moment the anchor commits, even against an operator who controls every private-chain replica. It is not a claim about the private chain’s contents being correct – only that they cannot change afterward without the change being visible to anyone holding the anchor.

2) The anchor frequency trade-off: AnchorPolicy.frequency is the cost/integrity lever: anchoring every block gives maximum integrity at maximum anchor traffic; anchoring every N th block cuts that traffic by $N\times$ at the cost of up to $N-1$ blocks’ unanchored tampering window. Table XI sweeps $N \in \{1, 5, 10\}$ across the paper’s own three cases, both chains, serialization on – the same condition §VIII’s comparison settled on.

Two findings, both against the frequency lever’s intuition. First, anchor overhead is at or below measurement noise, not a real structural cost: 15 of 18 cells fall within $\pm 1.4\%$, and the three outliers ($+9.37\%$, $+12.05\%$,

TABLE XI

Hybrid vs. private-only, summarized across both chains and all three cases per anchor frequency (18 raw cells: RESULTS.md M7-5, run 20260924T213216Z-2bd78940). Serialization on – the same condition §IV-F’s comparison settled on.

Anchor freq.	Overhead range	Anchors/chain	Bytes/anchor
Every block ($N=1$)	-0.37% to $+1.37\%$	5–15	≈ 1006
Every 5th ($N=5$)	-1.17% to $+12.05\%$	1–3	≈ 1006
Every 10th ($N=10$)	-0.54% to $+9.37\%$	1–2	≈ 1006

$+6.06\%$) do not track with frequency or chain in any consistent direction – the signature of scheduling jitter between two separately built clusters, not a per-anchor cost, and expected given `sync()/flush()` runs no consensus at all. Second, anchor chain storage is tiny and flat: every anchor costs ≈ 1.00 – 1.01 KB regardless of case or chain (5029/5, 10060/10, 15105/15 bytes-per-anchor all ≈ 1006), because an AnchorRecord is two 32-byte hashes plus fixed-size metadata, never a function of the private block’s own payload size. The frequency lever therefore trades anchor count linearly against the tampering window it leaves open, not against a real cost curve on the anchor side – the anchor’s own cost is already negligible at $N=1$.

3) What external verifiability buys: three security tests: Three tests, mirroring tests/unit/test_hybrid_chain.py, demonstrate what the anchor adds and where it honestly does not reach.

(a) Tamper a private block after it is anchored. A private-chain block already covered by an AnchorRecord is replaced, on every replica, by a different, internally well-formed, validly-signed block at the same height – the same class of attack §VIII’s Byzantine-leader test constructs, applied here to storage rather than to consensus. `verify_anchor` recomputes the live block’s hash and Merkle root on every call and compares them against the stored record: `anchored=True`, `verified=False`. The tamper is caught.

(b) Tamper a private block that has not been anchored yet. At frequency $N=5$, heights 6 and 7 of a 7-block chain are not covered by any anchor until a later multiple or a flush closes the window. Tampering height 6 in that gap produces `anchored=False`: there is nothing to check it against, so nothing is claimed either way. This is stated as the honest cost of infrequent anchoring, not smoothed over – it is exactly the $N-1$ -block tampering window named above as the trade-off’s price, demonstrated rather than assumed.

(c) Tamper the anchor chain itself. The anchor chain’s own stored blocks are replaced the same way as in (a). `verify_anchor_chain_integrity` – `Chain.verify_integrity`, unmodified, applied to the anchor chain – catches the broken link and the invalid signature, the same way it would for either private chain. This catches storage-level tampering of the anchor chain, but rests on an assumption stated plainly rather than hidden: it does not model an attacker who controls the anchor authority’s own signing

key from the start. In production the anchor chain is a real public blockchain with its own, separate consensus; an operator who also controlled that consensus would defeat the entire hybrid design, not just this check.

4) Limitations, stated honestly: The anchor chain here is simulated, and three simplifications matter for anyone reading these numbers as evidence about a real deployment. First, a real public chain (Ethereum, Bitcoin, or a purpose-built one) has real network latency and a real per-transaction cost – gas fees on Ethereum, most concretely – neither of which this simulation pays; the ≈ 1 KB/anchor figure above is a storage cost, not a gas estimate, and the two do not convert directly. Second, this simulation’s anchor authority is a single signing key with no consensus of its own; a real deployment anchoring to a public chain submits one signed transaction per anchor to that chain’s own, already-running consensus, which is a fair analogue of what is modelled here, but the public chain’s own liveness and finality guarantees are not exercised by anything in this section. Third, and most fundamentally, the paper’s threat model (§II) never specifies who the external verifier actually is – a regulator, a patient, an independent security researcher auditing the deployment – and this design does not pick one either. What it provides is that verification is possible for any of them without a relationship with the private chain’s operators, which is the property §IV-A’s trade-off was missing; deciding who should exercise it is a policy question this report does not answer.

E. Honeypot Data Poisoning: The Attack

§V-G’s Gap 1 named, without measuring, this framework’s sharpest irony: a Tier-2 adversary who controls the honeypot’s collection process can feed false training data into Sig_{RW}/FT_{RW} , and once that data commits to $BC_{Sig_{RW}}$ through real pBFT consensus, the chain’s own immutability – the paper’s central selling point – certifies the poison as tamper-proof, not as false. This section measures both halves of that claim.

1) The attack: three strategies, one adversary: A Tier-2 adversary controlling one cloud server that contributes to $BC_{Sig_{RW}}$ can submit crafted samples. Three strategies (detection.poisoning), each a pure transform on the training draw, never touching the eval set:

- Label flipping. Relabel a fraction of real ransomware training rows as benign. Features untouched – the “benign” contribution is a real ransomware trace under the wrong label.
- Feature poisoning. Inject synthetic rows labelled ransomware whose features are resampled from real benign training rows.
- Anchor-point injection. Inject rows interpolated at the midpoint between the benign and ransomware per-feature centroids, labelled benign – teaching the model that the class boundary itself is benign, pushing the effective boundary toward the ransomware

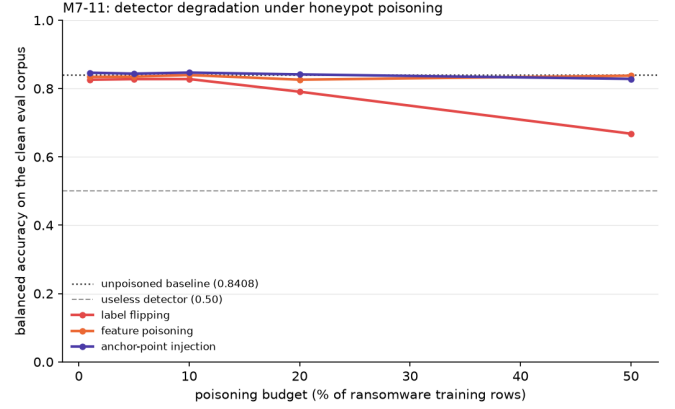


Fig. 2. Detector degradation under three poisoning strategies, five budgets each.

centroid without fabricating an obviously wrong vector or flipping any real label.

Budget is a fraction of the training set’s ransomware row count (717 of 1467 rows), swept at 1/5/10/20/50%, against the committed corpus, sanity-checked at the CSV-path baseline 0.8408 (not the chain-path 0.8422; DEV-27/DEV-37 explain the 0.0014 gap).

2) The degradation curves: sophistication is not effectiveness: Label flipping is the most damaging strategy, and the only monotonic one. -1.5 pt balanced accuracy at 1% budget, widening to -17.3 pt at 50%, collapsing to exactly $\text{bal_acc} = 0.5000$ at 100% – at full budget the training set has zero positive rows left, so no classifier can be fit past “always benign,” which scores 0.50 on a balanced eval set by construction, not by any model actually running. This matches the session brief’s own prediction: the simplest attack is the one most likely to succeed.

Feature poisoning is mild and roughly budget-insensitive (-0.3 to -1.5 pt across the whole range, non-monotonic). Duplicating real benign feature vectors under the ransomware label dilutes $AProf$ without teaching it anything structurally new – the injected rows are, feature-wise, indistinguishable from rows already in the benign class.

Anchor-point injection – billed as the most sophisticated strategy – is the least damaging at low-to-medium budget, and briefly improves accuracy: $+0.6$ pt at 1%, still $+0.1$ pt at 20%, only turning negative at 50% (-1.3 pt). Points at the inter-class midpoint, labelled benign, act as a mild regulariser at low budget rather than the intended boundary shift, before enough of them accumulate to move it. A designed attack’s sophistication and its measured effectiveness are not the same axis – the crudest attack (mislabel real data) beat the most carefully engineered one across most of the tested range.

3) The permanence argument, demonstrated once, for real: Not simulated:

scripts/m7_11_honey_pot_poisoning.py builds one real malicious CleanSample through the same honey_pot collection path every other record in this project uses, signs and attests it normally, then constructs its SignatureRecordPayload with label="benign" instead of the sample's true "RW" – label flipping, applied concretely to one record – and commits it through framework. phase2_collection's unchanged pipeline to a real BC_{SigRW} cluster running actual pBFT consensus. It committed in one block, cleanly, on the first attempt.

Algorithm 2 has no counter-move, and this is a code-level fact, not an inference: reading blockchain.chain.Chain's full public method list – append, check_append, create_genesis, adopt_genesis, draft_next, and read-only accessors – finds no delete, remove, rollback, revert, truncate, or undo method anywhere. No code was written to produce this finding; the class was already in the repository (OUT OF SCOPE for this session: no changes to the chain).

4) The detection gap: pre-commit and post-commit: Pre-commit. The poisoned record passed all five of Chain.check_append's checks – prev_hash linkage, Merkle root recomputation, hash uniqueness, ECDSA signature, timestamp-skew tolerance – every one structural or cryptographic, none semantic. A correctly formatted, correctly signed poisoned sample passes every existing validation check, because nothing in Algorithms 1-5 was ever asked to check semantic content, and this project's implementation does not accidentally add that check either.

Post-commit. Not built, and named as a gap rather than fixed. Honest nodes independently maintaining their own feature distributions and flagging statistical outliers in new contributions could in principle catch anchor-point-style poisoning (it visibly clusters near the class boundary) but not label flipping (its features are real; only the label lies) – and the mechanism itself would need cross-node feature-space agreement, a research problem this project does not attempt to solve.

5) Hybrid anchoring does not help: §V-D's anchor chain verifies that already-committed BC_{SigRW} data has not been tampered with after commitment – an integrity guarantee over time. Poisoning is not post-commit tampering; it is a validly signed, validly consensus-approved commitment by an authorized node in the first place. The anchor faithfully records the poisoned block exactly as submitted. It cannot distinguish a legitimate record from a false one an authorized party chose to submit, because that distinction is validity, not integrity – §V-G's Trust Assumption 2 already states the anchor chain's guarantee stops there.

6) Not a BSFR-SH-specific bug: Any system that trains a model on data from a source it does not fully trust, and commits that data to tamper-evident, append-only storage, faces the same structural tension: the storage layer's integrity guarantee and the training data's trustworthiness

are orthogonal properties, and strengthening the first – more replicas, stronger consensus, longer chains – does nothing to strengthen the second. Blockchain-backed ML training pipelines inherit this tension by construction, not by implementation mistake. It is the same "chain integrity is not chain validity" distinction §V-G's coverage matrix already needed for $SigRW/FTRW$'s own integrity-vs-truth caveat – Gap 1 is that distinction's sharpest consequence, not a new one.

F. Statistical Poisoning Detection: The Defense

§V-E measured the attack Gap 1 named: a validly-signed poisoned $SigRW$ record commits to BC_{SigRW} exactly as cleanly as an honest one, because Chain.check_append's five checks are all structural or cryptographic. This section builds and measures the defense Gap 1 predicted the shape of but never implemented – a genuine research contribution, not a reproduction of the paper or a critique of it, since Algorithm 2 has no semantic validation step to reproduce or critique in the first place.

1) The mechanism: batch-level drift, gated into the existing vote: detection.drift.DriftDetector keeps a per-replica running per-feature profile (RunningStats, Welford's online algorithm [11] – no historical sample is ever stored or re-read) and scores an incoming batch two ways: Mahalanobis, a diagonal, RMS-aggregated standardized distance of the batch centroid from the running mean; and Page-Hinkley [12], a classical streaming mean-shift test applied to that same scalar signal, for slow cumulative drift across many small batches. A full per-feature Kolmogorov-Smirnov test needs the historical sample values, not just their moments, so it was scoped out rather than built. This is deliberately not per-sample outlier detection – DEV-26 already rules that out for the honey_pot's own design, since a single unusual sample can be genuine, newly-observed ransomware behaviour, not poison.

consensus.validated_commit.ValidatedSigRWChain subclasses Chain and overrides check_append() – the one place consensus/pbft.py's own docstring says block validity is decided ("Consensus decides whether, Chain decides valid") – to run the detector over the block's decrypted $SigRW$ batch after the five existing checks pass. A replica that detects drift raises ChainError and is rejected by the existing pBFT vote-counting exactly like a bad signature would be: no new consensus message, no change to Replica's state machine. Cluster.__init__ gained one additive, backward-compatible chain_factory parameter (default Chain) so BC_{DTBU} 's commit path is unaffected.

2) Defended vs. undefended: all 15 of §V-E's cells: The defense catches the strategy predicted least catchable, and misses the one predicted most catchable. Anchor-point injection's interpolated class-boundary midpoint coincides, in a roughly class-balanced corpus, with the

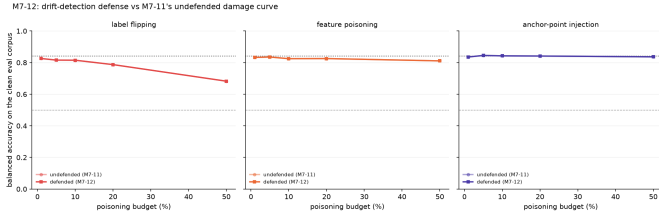


Fig. 3. Defended (Mahalanobis-gated) vs. undefended balanced accuracy, all three strategies, five budgets each.

population mean – it scores below the same-size clean-batch noise floor at every budget measured (e.g. 0.008 vs. a clean p_{95} of 0.11 at $n=143$), the sharpest measured case yet of “sophistication and detectability are not the same axis.” Label flipping and feature poisoning score above the matched noise floor at budgets $\geq 5\%$ – not because the detector reads labels (it structurally cannot; FT_{RW} and label are separate fields by design), but because an all-malicious batch’s feature composition differs from the profile’s usual mix. An honest, unusually ransomware-heavy batch – a real outbreak, not an attack – would trigger the identical alarm.

At the threshold this session shipped (Mahalanobis $RMS \geq 3.0$, chosen before any of the above was measured and left unmoved after), 0 of the 15 cells were actually prevented: every defended-column number in Table XII equals the undefended one, because nothing crossed the bar. This is not “the defense does not work” – see the false-positive analysis below for why the bar was not simply lowered.

3) False-positive rate, and the new-family tension: Zero of 30 chunked, never-poisoned eval-corpus batches (25 rows each) triggered a false alarm at the shipped threshold. A documented, disclosed feature-space perturbation of real ransomware rows – scaling network/beacon features up and crypto/encryption features down, modelling a hypothetical family that exfiltrates rather than encrypts, never a new call into honeypot/collector.py’s own generator – scored 1.2255, roughly $17\times$ its own matched clean-batch p_{95} (0.070), and was still not flagged at the shipped threshold. This is the tension item 4 of the M7-12 brief asked for, measured rather than assumed: a threshold loose enough to catch that probe sits close enough to the label-flip/feature-poison noise range that it would also raise the false-positive rate on ordinary small clean batches – the exact trade-off a real deployment would have to accept explicitly, not one this session resolved by picking a more flattering number.

4) The architectural observation, and its cost: Gating the check inside `check_append()` is the right integration point because it is the last moment before immutability makes the decision permanent – once committed, BC_{SigRW} ’s own guarantee (§V-E) prevents removal. Reusing pBFT’s existing $2f+1$ threshold, rather than inventing a new vote type, means a Tier-2 adversary who

also controls one consensus replica still cannot force a commit past three independently-refusing honest replicas.

That integration point has a cost this project had not paid before: blockchain/transaction.py’s own docstring states “miners validate blocks they cannot read; that is the point” – $SigRW$ payloads are hybrid-encrypted to one recipient (DEV-01), and no replica held that key before this session. ValidatedSigRWChain requires one, which §V-G states as a new trust assumption (TA-6) rather than a detail: this defense narrows a confidentiality invariant that held since M2b. A second new assumption (TA-7): every honest replica must run the identical DriftPolicy to agree on whether to vote – untested against a policy-mismatched replica, stated as a limitation rather than assumed away.

5) The gap is real, and this is a measured, partial answer to it: This defense is not in the paper’s design, and its absence is a real gap – §V-E demonstrated exactly how much that costs. It is also not a general fix: measured against the three strategies and five budgets already on record, it closes one case (a batch introducing feature values the corpus profile has never produced, submitted at enough scale against a mature history) and does not close the strategy M7-11 found most damaging, except as a side effect of batch composition an honest outbreak would trigger identically. A single record – §V-E’s own permanence-demo shape – is undetectable in principle at any threshold, since a batch of one has no averaging to suppress sampling noise. Reporting a solvable gap honestly, with the numbers that show exactly how far the solution reaches and where it stops, is the more useful contribution than reporting either “poisoning is unsolved” or “poisoning is solved.”

G. Coverage Matrix and Gaps

Three of the paper’s five properties hold at their defensible tier: (1) fully, (3) partially (Sybil and $f=1$, not the “51%” framing), (5) unconditionally. Two have measured limitations the paper does not acknowledge: (2) was never implemented, so it is unverified rather than disproven – there is no registration-authority module in this codebase to test it against; (4) is true for manipulation but has a demonstrated DoS gap the paper never tests for (DEV-20’s reduced view change: one lagging honest replica plus one Byzantine replica stalls the chain at $n=4$, demonstrated in `test_pbft_byzantine.py`). The sixth row is not a §V claim at all – the clearest form GAP-6’s “no formal model” takes: a threat model that never mentions adversarial ML cannot be faulted for failing against it, but a deployment relying on §V’s five properties alone would have no way to know that attack surface existed, since Algorithm 3 (§IV-C) assumes FT_{RW} is simply observed and never adversarially shaped.

Gaps the matrix surfaces, stated as findings, not fixed here. Data poisoning of the honeypot (Tier 2-adjacent, measured in §V-E): a compromised honeypot

Strategy	Budget	Mahalanobis score	Clean-batch p_{95} (matched n)	Prevented
Label flip	1–50%	0.32–0.57	0.24–0.55	No
Feature poison	1–50%	0.28–0.50	0.24–0.55	No
Anchor injection	1–50%	0.008–0.016	0.067–0.55	No

TABLE XII

Score ranges across all five budgets per strategy (full per-cell numbers: RESULTS.md M7-12). Label-flip/feature-poison scores exceed the matched noise floor at $n \geq 36$ (budgets $\geq 5\%$); anchor injection never does.

TABLE XIII

Coverage matrix: §V’s five claims, mapped onto the adversary tier(s) they actually address (§V-A), our verification method, and the result.

§V claim (paraphrased)	Tier	Method	Result	Known limitation
(1) Session keys/mutual auth defeat replay, MITM, impersonation	1	Scyther (M7-1), symbolic verification	VERIFIED – secrecy, mutual auth, impersonation resistance, bounded + unbounded	Within-window replay outside Scyther’s scope (test_session_replay.py instead); perfect-crypto abstraction
(2) Credentials deleted post-registration defeat insider/stolen-verifier	insider	none – no registration-authority module exists	UNVERIFIED / NOT IMPLEMENTED	Nothing to check the claim against
(3) pBFT resists 51% attacks/selfish mining; Sybil handled	2, 3	test_pbft_byzantine.py ($f=1/f=2$), Sybil test	MIXED: Sybil $f=1$ VERIFIED; “51%” framing WRONG (FLAW-5) – $f=2$ (50%) FORKS	“Selfish mining” has no pBFT analogue
(4) Blockchain immutability resists DoS, manipulation, leakage	1–3	Chain hash-linking + quorum (manip.); view-change analysis (DoS, untested by paper); hybrid encryption (leakage)	MIXED: manipulation VERIFIED, Tier 1–2; DoS has a MEASURED GAP; leakage PARTIAL (payload encrypted, meta-data not)	DoS-resistance never tested by the paper itself
(5) Two chains isolate detection from recovery (new, not in the paper) Feature-space evasion resistance	n/a 4	test_chains_independent.py (12 tests) M7-3 (attack), M7-8 (defense)	VERIFIED, structurally MEASURED: 25% of samples trivially evadable; defense real but narrow, non-monotonic in training budget	None identified Does not exist as a §V claim at all

could feed false records into Sig_{RW}/FT_{RW} ; once committed, BC_{SigRW} ’s immutability makes the poisoning permanent and tamper-evident, not detectable – chain integrity certifies a record was written as submitted, never that what was submitted was true. Insider threat at the registration authority: §V-2’s deletion claim is unverifiable from outside the deleting party and nothing in this project implements the registration flow to test it against. Denial of service against consensus (Tier 2, measured, not closed): Raft (§V-C) is faster when honest but is worse under exactly this adversary – it loses safety, not just liveness, to a single Byzantine leader – so it is a different, worse trade-off, not an available fix. Side-channel leakage from the encrypted backup chain (Tier 1-adjacent): transaction sizes and timing on BC_{DTBU} are not encrypted, and an external network observer can infer backup frequency and rough data volume without decrypting anything, a structural property of committing variable-sized encrypted blobs to a chain whose block headers are, by the hybrid design’s own premise (§V-D), meant to be externally visible.

The paper argues five security properties without stat-

ing what adversary each defends against. We constructed the threat model the paper omits and mapped our verification results onto it. Three properties hold at their stated tier. Two have measured limitations the paper does not acknowledge, and a sixth attack surface – adversarial ML – exists that the paper’s own vocabulary has no way to name.

VI. Detection Analysis

§III-C designed the honeypot feature pipeline the paper omits. This section reports what it measures once built end to end and pressure-tested: the pipeline’s own result, whether it survives an adversary who knows a behavioural sensor is watching, whether hardening it against that adversary is worth its cost, and whether it transfers to real, never-synthetic malware data.

A. Honeypot Pipeline Result

§III-C described the design of the honeypot feature pipeline the paper omits. This section reports what it measures once built end to end: honeypot deployment → Phase-2 collection → real four-node pBFT consensus onto BC_{SigRW} → Phase-3 decryption, training, and detection

TABLE XIV

Honeypot detection result (Alg. 3 end to end) and its derived confusion matrix. TP/FP are computed from the reported precision, recall and evaluation-set composition (353 of 731 eval records positive) – a computed quantity, not a separately measured one.

Train n / Eval n	1467 / 731
Balanced accuracy	0.8422
Precision	0.8439
Recall	0.8272
MCC	0.6849
PR-AUC	0.9100
Constant-positive baseline	0.5000
Derived confusion matrix, eval set ($n = 731$)	
True positive	292
False negative	61
False positive	54
True negative	324

– the sequence Fig. 3 of the paper draws and the paper’s own evaluation never runs.

Table XIV reports the result on the committed corpus (1467 training records, 731 evaluation records, drawn from two independent seeds so no latent generator parameter is shared across the split): 0.8422 balanced accuracy, between the measured simple-Gaussian baseline (0.830) and the corpus’s stated intended ceiling (≈ 0.85 , §III-C) – above the naive baseline, not above the ceiling that would indicate the classifier is reading a leak rather than the corpus’s designed difficulty. The evaluation script checks this automatically against a 0.05 leak margin and refuses to record a result that would fail the check; it did not fire here. The derived confusion matrix shows a detector that is neither trivially accurate nor degenerate: 292 of 353 ransomware records correctly flagged, 61 missed, 54 benign records misclassified.

This is the first number in this project produced by Phase 2’s output feeding Phase 3’s input through the paper’s own architecture. It demonstrates that the framework, as specified, is not merely completable on paper – once §IV-B’s undefined Sig_{RW}/FT_{RW} are actually given a definition, Algorithms 2 and 3 connect into a working, non-degenerate detection pipeline. This is the strongest available rebuttal to a reading of FLAW-2 as “the framework as designed cannot work”: it can. The paper simply never tried.

B. Adversarial Robustness: The Attack

§VI-A’s 0.8422/0.8408¹ balanced accuracy answers “does the detector work.” It does not answer “does it keep working against an adversary who knows a behavioural

¹This section trains and evaluates on data/honeypot/corpus_{train,eval}.csv directly, per §III-C’s “committed corpus is the fixed dataset” convention. Read this way the same seed and configuration give 0.8408, not 0.8422: honeypot/corpus.py formats every feature to six significant figures when it writes the CSV, and that rounding alone accounts for the 0.0014 difference from Table XIV’s chain-reconstructed, full-precision number. Both are measured; neither is wrong; every threshold below is relative to 0.8408.

TABLE XV

Top-5 features by Random Forest importance (individually, bal. acc. 0.8513 on the committed corpus – the higher of the two importance-bearing models, Decision Tree scoring 0.8063), with each feature’s evasion direction and physical bound.

Feature	Importance	Direction	Bound
observed_stages	0.126	↓	0 stages
extension_change_rate	0.109	↓	0
rename_rate_per_s	0.079	↓	0 s ⁻¹
write_entropy_var	0.074	↑	6.5494
crypto_ngram_novelty	0.067	↓	0

sensor is watching.” The paper’s threat model is silent on this: Algorithm 3 assumes FT_{RW} is simply observed, never that a rational attacker – one motivated to avoid exactly the detection Algorithm 3 performs – would adjust their program’s observable behaviour to defeat it. This is not a gap we can hold against the paper alone: testing it requires a defined, differentiable feature pipeline, which is precisely what §IV of the paper never provides (FLAW-2). Only having built FT_{RW} ourselves (DEV-03, DEV-27) makes the question answerable at all.

Table XV’s bounds are not arbitrary. Four features move toward 0 – an attacker can always throttle: delay the kill chain so fewer of the four observable stages complete inside a honeypot’s finite dwell time (observed_stages), rename files less aggressively or not at all (extension_change_rate), slow the rename loop (rename_rate_per_s), or mimic a common, previously-seen crypto-API call sequence instead of a distinctive one (crypto_ngram_novelty). The fifth, write_entropy_var, moves the other way: ransomware ciphertext is consistently near-maximal entropy (low variance across blocks), while benign writes are heterogeneous, so an attacker raises variance by interleaving some deliberately low-entropy writes among the ciphertext. Its bound, 6.5494, is not an abstract limit but the highest write_entropy_var the generator’s own benign profiles are ever observed to produce in the committed corpus – an attacker cannot manufacture more heterogeneity than a real, non-malicious write pattern in this environment exhibits. No feature in the top-5 turned out structurally robust (unlike mean write entropy, which cannot fall below the entropy of properly encrypted data and is exactly the kind of feature that would resist this attack, but did not rank in the top 5).

Single-feature evasion is weak. Figure 4 shows every individual feature, pushed all the way to its bound, costing the detector at most 9 points of balanced accuracy (0.841 $\rightarrow$ 0.752 for write_entropy_var, the largest single-feature drop; rename_rate_per_s alone costs under 3 points). None approaches 0.50. A one-dimensional attacker – one who tunes a single observable behaviour and leaves everything else alone – does not meaningfully dent this detector.

Combined evasion compounds, but does not collapse the

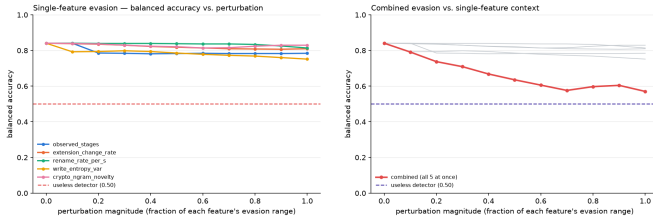


Fig. 4. Left: each top-5 feature perturbed independently, 0–100% of its evasion range, against the committed eval corpus’s 353 malicious rows (ensemble fit once, never retrained). Right: all five perturbed simultaneously (**bold**), against the five single-feature curves as context (light gray) on the same axes.

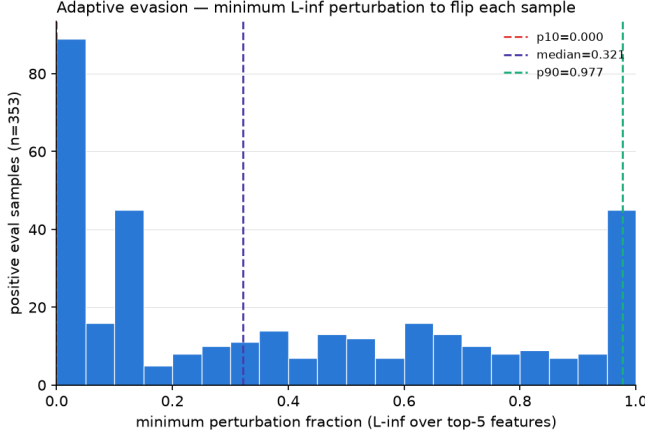


Fig. 5. Per-sample minimum L_∞ perturbation (top-5 features, each normalized to its own evasion range) that flips the ensemble’s verdict, over all 353 positive eval rows.

detector. Perturbing all five features together (Figure 4) drops balanced accuracy from 0.841 to a floor around 0.57–0.60 by 60–100% joint perturbation – roughly triple any single feature’s damage, confirming the features compound as a competent adversary would exploit. It never reaches the 0.50 useless-detector line, though: the top-5 features carry only 45.5% of Random Forest’s total importance mass ($0.126+0.109+0.079+0.074+0.067$), so the remaining 17 features, entirely unperturbed here, still carry real discriminative signal even once the five most important ones are pushed to their physical limits. This is itself the finding – exactly the outcome the session brief’s own test criterion anticipated as a possibility (“the model uses features outside the top-5 enough to still discriminate”), and it is a mild point in the detector’s favor: no small, named feature subset is a single point of failure.

Adaptive, per-sample evasion is a different picture. An adversary who knows the model and optimizes per sample does much better than one perturbing blindly. Binary search per positive row over the L_∞ ball (Figure 5) finds: median minimum perturbation 0.321 (a typical ransomware sample needs about a third of the available evasion range to flip); 10th percentile 0.000 – driven by

the 61 of 353 positive rows (17.3%) the detector already misses at 0% perturbation (consistent with the measured 0.8408’s own false-negative rate), plus a further 7.9% needing under 5% perturbation, so roughly a quarter of ransomware samples are trivially evadable by an adaptive attacker; 90th percentile 0.977 – the hardest samples need nearly the entire evasion range. 28 of 353 positive rows (7.9%) never flip even at the full L_∞ ball. Checking those survivors against the sixth-through-tenth ranked features (never perturbed here) shows exactly what the session’s own test criterion predicted: survivors average `c2_beacon_count` 10.3 against 7.0 for the malicious class as a whole, `entropy_delta` 3.58 against 2.61, and similarly elevated `key_generation_events`, `shadow_copy_deletions` and `dns_entropy` – these are the ransomware samples so extreme on features outside the top-5 that disguising only the top-5 cannot save them.

Conclusion: shape (c), the nuanced answer. No single top-5 feature is fragile enough to break the detector alone, and no combination of exactly these five is either – the ensemble’s reliance on 17 further features is real and measured, not assumed. But a median-effort adaptive adversary needs only about a third of the physically plausible evasion range, and roughly a quarter of ransomware samples are evadable with under 5% perturbation on features an attacker can freely throttle to zero. The paper’s implicit assumption that FT_{RW} is unconditionally robust is false for a meaningful minority of cases and approximately true for the rest, and this asymmetry – some samples essentially free to evade, others requiring near-maximal effort – is exactly the kind of result a single aggregate accuracy number cannot show. It is a direct consequence of FLAW-2 (§VIII-A): the paper’s own evaluation, run on BitcoinHeist address features that no attacker can choose their own value of at will, could never have surfaced this question, because it is a question about behavioural features an attacker actually controls. Only the framework path this project built makes it a question with a measured answer.

C. Adversarial Robustness: The Defense

§VI-B measured the attack. This section measures a candidate defense: if the ensemble is fit on a training draw that already contains perturbed positives, do the fragile samples become robust, and what does that cost on clean data? The training draw’s 717 positive rows are each augmented with $K = 5$ perturbed copies, one independent $\text{Uniform}(0, \text{budget})$ fraction per top-5 feature per copy, moved toward that feature’s own evasion bound (Table XV) – negative rows and the eval corpus are never touched. Three training budgets are swept – 25%, 50%, 100% of each feature’s evasion range – plus a 0% degenerate case as a pure sanity check. All perturbation and scoring machinery is identical to §VI-B’s (‘detection.adversarial’, factored out this session so both

TABLE XVI

Clean-eval cost of adversarial retraining. Baseline: bal. acc. 0.8408, precision 0.8415, recall 0.8272, MCC 0.6822.

Budget	Bal. Acc.	Δ	Prec.	Rec.	MCC
25%	0.7803	-6.05pt	0.886	0.637	0.588
50%	0.7616	-7.92pt	0.845	0.632	0.544
100%	0.7363	-10.45pt	0.828	0.586	0.498

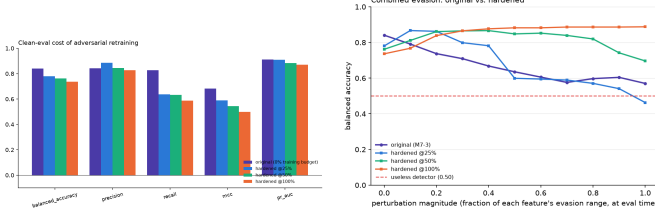


Fig. 6. Left: clean-eval metrics, original vs. hardened at each training budget. Right: combined evasion (§VI-B’s attack) against the original model and each hardened model.

experiments share one tested implementation rather than two copies that could silently drift).

The degenerate case reproduces the original model exactly. At 0% budget, augmentation is a no-op by design (‘detection.retraining.augment_positive_rows’) rather than five duplicate copies of every positive row: duplicating would still change RandomForestClassifier’s bootstrap sampling for zero new information, breaking bit-exact reproduction. Measured: train_n=1467 (unchanged), bal_acc=0.8408, identical to §VI-B’s baseline to the last recorded digit.

The cost is real and grows monotonically with training budget (Table XVI, Figure 6), driven almost entirely by falling recall ($0.8272 \rightarrow 0.586$ at 100%) rather than rising false positives – precision stays high throughout. The hardened models are not noisier; they are more conservative, increasingly refusing to call genuine, unperturbed ransomware “ransomware.”

Robustness is non-monotonic in training budget – the headline finding. At 25% budget, the hardened model is worse than doing nothing: median adaptive perturbation to evade falls from the original’s 0.3213 to 0.2499, and the combined-evasion curve (Figure 6) crosses the 0.50 useless-detector line at full perturbation (0.463), something the original model’s own floor (0.570) never did. A narrow training budget teaches the model the small-perturbation region at the cost of the large-perturbation region it never saw during training – hardening that backfires against a stronger attacker than the one it was tuned against. At 50% and 100% budget the opposite happens, sharply: median adaptive perturbation rises to 0.8825 and 1.0000, and the 100%-budget combined-evasion curve increases with perturbation instead of falling (0.736 at 0% eval-time perturbation $\rightarrow$ 0.888 at 100%, Table XVII). The mechanism: a large fraction of the hardened model’s own positive training rows sit at or near the top-5 features’

TABLE XVII

Adaptive evasion: minimum L_∞ perturbation to flip, original vs. hardened.

Model	p10	median	p90
original (§VI-B)	0.000	0.321	0.977
hardened @25%	0.000	0.250	0.951
hardened @50%	0.000	0.883	1.000
hardened @100%	0.000	1.000	1.000

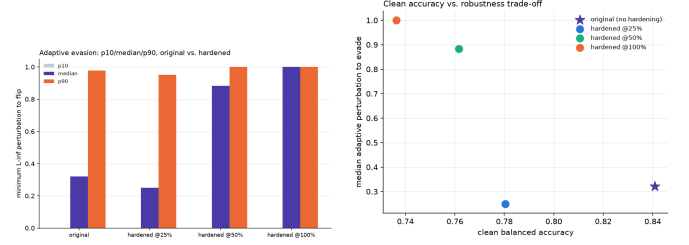


Fig. 7. Left: p10/median/p90 minimum perturbation to evade, original vs. each hardened budget. Right: clean balanced accuracy vs. median adaptive perturbation to evade – the deployment trade-off, one point per training budget.

evasion bounds, so the model learns “values at the bound” as a strong ransomware signature in its own right. That is genuine, measured robustness against §VI-B’s specific attack shape – but it is pattern-matching on the augmentation’s own footprint, not a deeper representation, and is untested against any attack that does not resemble “features pushed toward their historical evasion bound.”

The “throws away signal” failure mode occurs, partially. The session’s own test criterion: does the hardened model solve robustness by driving top-5 importance to near zero rather than learning a robust representation? Top-5 importance sum falls from 0.4553 (original) to 0.4044 (25%), 0.2966 (50%), 0.2309 (100%) – a real, budget-proportional redistribution, never collapsing to near zero. The features consistently gaining importance – directory_breadth, entropy_delta, key_generation_events – are none of them perturbed in this experiment. This is the partial case the test anticipated: not “solved robustness by discarding the top-5 entirely,” but a real, measured shift of reliance onto features an attacker could target next, which a follow-up session would need a new perturbation round against the hardened models’ own new top-5 to test.

The Pareto trade-off is not a smooth curve. Figure 7: 25% budget is strictly dominated by the original model (worse clean accuracy and worse robustness); only 50% and 100% trade clean accuracy for robustness in the direction a defender would want, and 100%’s marginal robustness gain over 50% (median 1.000 vs. 0.883) is small next to its additional clean-recall cost (0.586 vs. 0.632 recall) – 50% budget is the more defensible operating point of the three tested, not 100%.

Conclusion: adversarial retraining is a real but narrow, expensive defense. At a well-chosen budget it measurably hardens the detector against §VI-B’s specific attack –

TABLE XVIII

Adaptive evasion (median) by training budget, RF vs. MLP. RF's own numbers reproduce §VI-C; the MLP's 25% row is the headline contrast.

Budget	RF median	MLP median
original	0.3213	0.4866
25%	0.2499 (worse)	1.0000
50%	0.8825	1.0000
100%	1.0000	1.0000

median evasion effort rises from needing a third of the evasion range to needing nearly all of it. But every budget that helps costs 8-10 points of clean balanced accuracy, driven by recall, meaning a hardened deployment would silently miss more real, unperturbed ransomware in exchange for resisting an adversary sophisticated enough to know the top-5 features and their physical bounds. And the mechanism behind the gain – learning that near-bound feature values are themselves suspicious – is exactly the kind of narrow, augmentation-shaped pattern that a differently-shaped attack (partial perturbation of a different feature subset, or of the newly-important `directory_breadth/entropy_delta/key_generation_events`) was never tested against and has no particular reason to survive. Whether this trade-off is worth taking is a deployment decision this project can inform but not make: it depends on whether an operator weighs a sophisticated feature-space adversary more heavily than routine, unperturbed ransomware, which this framework's own honeypot corpus (DEV-27) was built to represent as the common case. The more durable fix implied by both §VI-B and this section is architectural, not just training a wider draw – ensemble diversity over feature subsets, or a loss that penalises reliance on any single small feature group, neither of which this session's scope covers.

1) Neural vs. Tree-Based Detector: Architecture or Features?: §VI-C's memorisation failure mode (the 100%-budget model learns “values near the evasion bound” well enough that its combined-evasion curve rises with perturbation) – is that a property of tree ensembles' axis-aligned splits, or of the feature space and augmentation strategy, independent of architecture? This section runs the identical protocol – same top-5 features and evasion bounds, same training budgets, same downstream detector/adversarial/retraining machinery – against a small MLP (detection.mlp_model, DEV-36: two hidden layers of 32/16 units, adam, L2 regularisation and early stopping) plugged into the same NProf/AProf profiles as a single-model ensemble. Feature importance is compared by permutation (an MLP has no Gini-style importance), so §VI-C's Gini-based top-5 sums are not directly comparable even for the same RF model, recomputed here like-for-like.

Clean accuracy is comparable (RF 0.8408, MLP 0.8237) and, before any hardening, the MLP is already more robust. Its median adaptive-evasion perturbation is 0.4866 against RF's 0.3213 – a real architectural difference: a

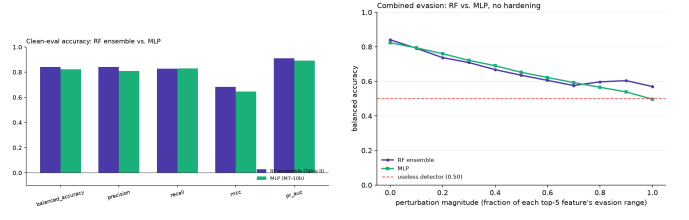


Fig. 8. Left: clean-eval accuracy, RF ensemble vs. MLP. Right: combined evasion, RF vs. MLP, no hardening.

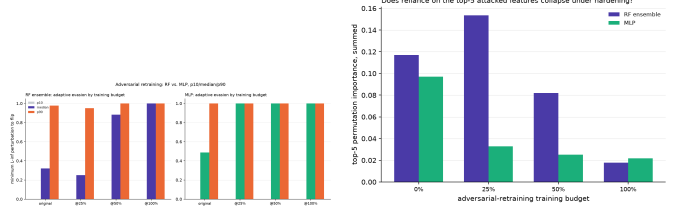


Fig. 9. Left: p10/median/p90 adaptive evasion by training budget, RF vs. MLP. Right: top-5 permutation-importance sum by training budget, RF vs. MLP.

smooth, distributed decision boundary needs a larger combined push across all five top features than a forest of axis-aligned splits does. Both models' unhardened combined-degradation curves (Figure 8) fall smoothly with perturbation in the ordinary way (RF: 0.841→0.570; MLP: 0.824→0.497) – neither shows the rising-curve memorisation signature before retraining.

The headline result: the MLP shows §VI-C's memorisation failure mode earlier and more completely than RF, not less. Every one of its three hardened combined-degradation curves rises monotonically with perturbation – 25%: 0.828→0.877; 50%: 0.810→0.897; 100%: 0.770→0.896 – the same signature §VI-C found only at RF's 100% budget, here at every budget tested. Its median adaptive-evasion perturbation jumps to 1.0000 at 25% and stays there (Table XVIII), and critically it never reproduces RF's 25%-budget backfire (RF's median evasion effort drops to 0.2499 before recovering; the MLP's never dips). A higher-capacity, continuously-weighted model does not avoid the augmentation-boundary-memorisation failure mode – it engages that shortcut faster and more uniformly than an ensemble of axis-aligned trees does.

Feature-importance shift, on different schedules. By permutation importance (Figure 9), the MLP's top-5 reliance falls monotonically and sharply with budget – 0.0971 → 0.0328 → 0.0251 → 0.0219, a 77% reduction end to end. RF's own permutation-importance is non-monotonic: 0.1170 → 0.1535 (a rise at 25%) → 0.0818 → 0.0177 – consistent with, and helping explain, RF's 25%-budget backfire: by this measure the narrowly-hardened RF becomes more reliant on the attacked features at exactly the budget where it is also most fragile. Neither model solves robustness by discarding the top-5 entirely,

matching §VI-C’s “partial, not total” finding and extending it to the MLP.

Conclusion: the fragility is in the features and the augmentation strategy, not the tree structure. A tree-ensemble-specific artefact would predict a distributed neural representation responding differently to the same augmentation; it does the opposite, adopting the rising-curve signature at every budget including the one where RF instead backfires by a different mechanism. Both failure modes point to the same root cause: augmenting positives toward a bounded region of a fixed, low-dimensional feature space gives any sufficiently flexible model a shortcut easier to learn than the underlying behavioural signal. The MLP’s one genuine architectural advantage is orthogonal to hardening (a smoother decision boundary, already harder to evade before any retraining); the fix this section implies is the same one §VI-C already named – diversifying what the augmentation teaches – not a change of model family.

D. Real-Data Transfer: ClaMP and EMBER

Why real-data evaluation matters. M4b’s 0.8422 balanced accuracy (§VI-A) is the detection ensemble scored on honeypot.collector’s own synthetic corpus – the same session designed the generator and fit the classifier against it, so that number measures the generator-detector pair, not whether the detector would recognise anything the generator did not anticipate. This section asks the question synthetic-only evaluation cannot answer: does the detector transfer to real behavioural features it has never seen, from programs it was not designed around? The detector used below is the identical fitted ensemble – same seed, same committed corpus, never retrained.

Dataset. The session brief named EMBER, CIC-MalMem-2022/CICMalDroid-2020 and MalwareBazaar, in that order. EMBER’s actual feature dataset is a ~2.4GB archive; this sandbox’s measured throughput on a small test fetch (~40.6KB/s) projects a ~17-hour download, infeasible in one session. The CIC datasets are gated behind a research-access request form, not a scriptable download. Used instead: ClaMP (Classification of Malware with PE headers, github.com/urwithajit9/ClaMP) – 5210 real Windows PE files’ static header-derived structural features, 2722 malicious and 2488 benign, openly published for exactly this kind of ML research use. Fetched once and committed at `data/external/clamp/` (1.28MB, sha256 recorded in its README) for full reproducibility, and used for evaluation only: it never fits or tunes anything in honeypot/ or detection/.

The mapping table is itself the first finding. FT_{RW} (§III-C) is a dynamic behavioural schema – what a monitor observes a running program do. ClaMP is static – header fields read off a file that is never executed. Table XIX shows what that mismatch produces: only the entropy group (3 of 22 features) has any real correspondence at all, because entropy is one of the few

TABLE XIX
 FT_{RW} features mapped from ClaMP, by group. 19 of 22 features have no analogue in a static PE-header dataset.

Group	Features	Mapped (direct/proxy)	Missing
entropy	3	3	0
filesystem	5	0	5
crypto_api	3	0	3
process	3	0	3
network	3	0	3
persistence	3	0	3
kill_chain	2	0	2
Total	22	3	19

observables measurable both statically (a file’s own byte distribution) and dynamically (the byte distribution of what a process writes). The other six groups – filesystem, crypto API, process, network, persistence, kill-chain – have no analogue in a dataset that never ran the sample, and are marked missing, handled exactly as `honeypot.features.build()` already handles an unobserved sensor: value 0.0, missing_mask bit set.

Transfer result: not a partial degradation, a clean failure, and mildly worse than chance. Before touching ClaMP, the script reproduces the established CSV-path baseline exactly (0.8408, §VI-B’s sanity condition) as a check that the mapping code has not corrupted the fitted ensemble. Scored on all 5210 mapped ClaMP rows: `bal_acc` = 0.5000, `precision` = 0.0000, `recall` = 0.0000, `mcc` = 0.0000, `pr_auc` = 0.4612 (Figure 10). `bal_acc` = 0.5000 exactly is not a rounding coincidence: the ensemble predicts every row, malicious and benign alike, as benign. More strikingly, `pr_auc` = 0.4612 sits below this dataset’s no-skill baseline (class prevalence $2722/5210 = 0.5225$) – the continuous ensemble score is mildly anti-correlated with the true label here, not merely uninformative. The mechanism is legible: with 86% of the feature space collapsed to a mapping artifact of zero, there is essentially nothing left for the profiles fitted on the synthetic corpus to discriminate on beyond three entropy features, and a mostly-zero vector apparently reads as closer to *NProf* than *AProf* under those fitted distributions. This is a finding about the schema’s dynamic-vs-static mismatch with the most tractable available real dataset, not evidence that the synthetic generator’s own difficulty calibration is wrong (DEV-27’s 0.85 Bayes-optimal ceiling) – no dataset that never executed a sample could ground the other six feature groups, whatever the generator’s own distributions looked like.

Distribution comparison: which synthetic features match reality. Two-sample Kolmogorov-Smirnov, synthetic corpus vs. ClaMP-mapped, for the 3 features with a real analogue (Figure 10) – the other 19 are a constant 0.0 under the mapping, and a KS test against a constant characterises the mapping, not the generator, so it is not reported as a distributional claim:

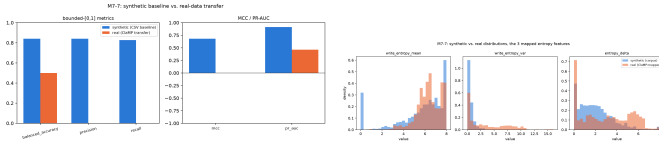


Fig. 10. Left: synthetic CSV-path baseline vs. ClAMP real-data transfer, same fitted ensemble. Right: synthetic vs. real (ClAMP-mapped) distributions, the 3 mapped entropy features.

Feature	D	p	syn. / real mean
write_entropy_mean	0.197	6.9×10^{-53}	5.63 / 6.36
write_entropy_var	0.488	$< 10^{-300}$	0.59 / 3.52
entropy_delta	0.285	1.7×10^{-111}	2.00 / 3.00

All three diverge significantly. Read cautiously, not as “the generator is wrong”: ClAMP’s entropy readings are whole-file and per-section entropy of arbitrary real PE binaries – many of them large, compiled, resource-laden files with high baseline entropy regardless of malice – while the synthetic features model entropy of write operations during a specific kill-chain episode, tuned to a 0.85 Bayes-ceiling difficulty budget for a deliberately hard classification problem. These are different measurement modalities compared on the closest available real analogue, not the same observable measured twice. One further caveat: $\sim 8\text{--}9\%$ of the synthetic corpus’s `write_entropy_mean` values are the generator’s own missing-value zero-fill, visibly inflating the low end of the synthetic histogram in Figure 10 and modestly affecting, but not creating, the measured divergence.

Limitations. The mapping is necessarily imperfect: static header fields are the closest real analogue available for entropy, and no real analogue exists at all for six of seven feature groups, so the effective comparison the detector faces is far narrower than the 22-dimensional space it was fit on. ClAMP’s own labelling process (VirusTotal-report-based, per its own documentation) may differ in criteria and era from the ransomware-specific threat model FT_{RW} ’s kill-chain groups were designed around – ClAMP’s malicious class is general malware, not specifically ransomware. The transfer gap measured here is therefore a statement about how far our generator’s approximation reaches when most of its feature space cannot be exercised at all, not a general claim about honeypot-based ransomware detection.

A richer static dataset: EMBER. EMBER2018 (github.com/elastic/ember) exposes far richer raw PE data than ClAMP – a 256-bin whole-file byte histogram, every section’s own entropy and read/write/execute flags, and a full per-DLL import table – and honeypot. `ember_mapping` (DEV-35) grounds 6 of 22 FT_{RW} features against it (entropy 3/3, `crypto_api` 2/3, `filesystem` 1/5), double ClAMP’s 3/22, verified line-by-line against the elastic/ember extractor source rather than assumed. The dataset (200,000 fully-labelled real PE files, sha256-verified) was scored against the identical fitted ensemble, evaluation-only, same posture as ClAMP.

TABLE XX
EMBER transfer vs. the ClAMP baseline (M7-7), same fitted ensemble.

Dataset	Mapped	Bal. Acc.	Prec.	Rec.	PR-AUC
ClAMP (M7-7)	3/22	0.5000	0.0000	0.0000	0.4612
EMBER (M7-10a)	6/22	0.5000	0.6667	0.00002	0.6301

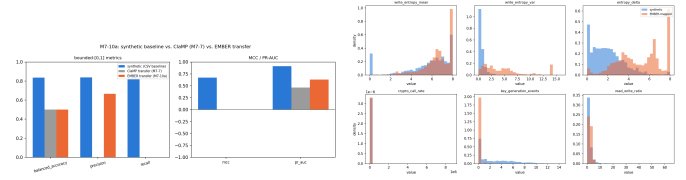


Fig. 11. Left: synthetic baseline vs. ClAMP (M7-7) vs. EMBER transfer. Right: synthetic vs. real (EMBER-mapped) distributions, the 6 mapped features.

Still exactly chance on the hard decision – but the continuous score is no longer anti-correlated with truth. Table XX: `bal_acc` = 0.5000 again, not a coincidence – the fitted NProf/AProf profiles place all but 3 of 200,000 rows on the benign side regardless of true label (2 of those 3 are true positives: recall 0.00002, precision 0.6667). The mechanism is the same one ClAMP exposed: 16 of 22 dimensions collapse to the structural-missing zero (missing_mask constant across every row), and profiles fitted where 16/22 features are normally populated read a mostly-zero vector as “benign” almost universally. What doubling the coverage fraction changes is the continuous score, not the hard call: `pr_auc` = 0.6301 sits above this dataset’s balanced 0.50 no-skill line, where ClAMP’s `pr_auc` = 0.4612 sat below its own 0.5225 no-skill line (M7-7) – EMBER’s ensemble score is weakly but genuinely informative about the true label; ClAMP’s was mildly anti-correlated with it. Richer static grounding measurably helps the model’s underlying confidence signal without moving the threshold decision it is discarded into.

Distribution comparison: EMBER diverges from the synthetic corpus more sharply than ClAMP did, on every mapped feature (Figure 11; full KS statistics in DEV-35 and RESULTS.md). The largest gaps (`write_entropy_var`, `crypto_call_rate`, $D > 0.6$) are partly a units mismatch rather than a substantive disagreement – the synthetic generator’s `crypto_call_rate` is drawn in calls-per-second units, EMBER’s mapped value is a raw import count, and only the two features’ presence is meaningfully comparable across that gap, not their raw scale.

The question this section set out to answer: does a richer static source close the schema’s dynamic-vs-static gap? No – 11 of 22 features (process, network, persistence, kill-chain) stay unmapped regardless of dataset richness, because no PE-header, import, or section field can observe a process spawn, a network connection, or a kill-chain stage; those require execution, which is what makes a

dataset dynamic in the first place. A richer static source narrows the coverage gap (6/22 vs. 3/22) and improves the discarded continuous score, without changing the shape of the gap or the hard-decision transfer outcome.

VII. Scalability Analysis

Two questions the paper’s own §VII benchmarks timing but never asks: how much on-chain storage does this design consume at realistic scale, and does its reported throughput actually reflect a system that scales.

A. Storage Cost (GAP-2)

Section VII benchmarks consensus timing but never asks how much storage its own design consumes, or at what scale that stops being feasible. We compute this from values already measured elsewhere in this project – the declared 4096B transaction payload (DEV-15), the measured 215B framing and 161B hybrid-encryption overhead per chunk (DEV-24), and the measured $1.14\times$ full-block overhead factor (header, Merkle root and canonical encoding, amortised over a 100-tx block, §IV-D) – rather than from new estimates; full arithmetic in docs/STORAGE_ANALYSIS.md.

At hospital scale, the paper’s own on-chain design fills 1 TiB per replica in about six months under daily backups; at network scale, in under three weeks. Backup frequency is the larger lever of the two the paper leaves open: at the medium scenario, daily backups cost $\approx 7\times$ weekly and $\approx 30\times$ monthly in chain growth for identical clinical content, a bigger effect than DEV-15’s payload-size sweep found from a $16\times$ range of payload sizes alone.

Against this, an on-chain-hash / off-chain-data design – store only a content digest and a storage pointer on-chain, keep the encrypted payload in external storage – preserves the same tamper-evidence the paper’s Merkle-root-and-signature chain already provides (recomputing and comparing the digest catches off-chain tampering exactly as the paper’s own immutability argument requires), at ≈ 504 bytes per backup regardless of payload size, against ≈ 1.14 MiB for a 1 MiB backup under the paper’s design – a $\approx 2,371\times$ reduction that widens further as backups get larger, since the hash-only cost is flat and the paper’s is not. What it does not preserve is on-chain availability: lose the off-chain store and the digest alone cannot reconstruct the backup, where the paper’s design always can. We state this trade-off rather than resolving it, per this project’s rule against silently revising the paper’s design.

Compute is not the bottleneck this analysis surfaces: the large-network scenario’s daily block count, at §IV-D’s measured marginal per-block cost, comes to $\approx 4,045$ CPU-seconds/day ($\approx 4.7\%$ of a day) on the same 8 GB dev laptop – comfortably inside its measured throughput ceiling, though nothing here measures a production cluster’s network or disk behaviour, only this one machine’s compute.

B. Throughput Does Not Scale With Chain Size (DEV-08)

§IV-D’s measurement is the scalability-relevant fact directly: per-block marginal cost is flat across cases 1 through 3 on both chains once cluster setup is excluded from the timed span (Table VII), and derived TPS is correspondingly flat rather than rising. A design whose own published throughput curve is the amortisation of a fixed setup cost over a growing block count, not a property of the consensus mechanism, does not get faster as the deployment grows – the six published TPS values equal $\text{total_tx}/\text{total_time}$ exactly to four significant figures, and removing the fixed cost from the timed span removes the rise, exactly what an amortisation account predicts and what §IV-F’s independently-measured Raft comparison confirms is not an artifact of pBFT specifically either.

VIII. Critique

Five defects, ordered by severity rather than by the order we found them. They are the ones that rise to a methodological or architectural problem; a complete record of every smaller departure from the paper’s published design, each with its own rationale, is maintained separately and is not reproduced here.

A. FLAW-2 – framework/evaluation disconnect

Algorithm 2 collects data from a ransomware honeypot and Algorithm 3 trains a detector on the resulting signatures and behavioural features – properties of an executing, or emulated, program. Section VII then evaluates entirely on BitcoinHeist: address-level graph features (length, weight, count, looped, neighbours, income) of a Bitcoin address that has already received a ransom payment. These are different problems. Detecting a running ransomware program from its behaviour and forensically labelling a wallet address after the fact share no features, no data-generating process, and arguably not even the same notion of a positive instance – one is prospective and behavioural, the other retrospective and financial. Nothing in §IV-C or §VII acknowledges the substitution.

We built both pipelines rather than choosing one. The BitcoinHeist pipeline (§IV-A–§IV-C) reproduces the paper’s actual published evaluation. The honeypot pipeline (§III-C, §VI-A) runs Algorithm 2’s honeypot output through real Phase-2 consensus onto BC_{SigRW} , decrypts it back out, and trains Algorithm 3’s four models on it – the sequence the paper’s own Fig. 3 draws. It produces a real, bounded result (0.8422 balanced accuracy, §VI-A). The paper’s framework half, run end to end as specified, was never demonstrated by its authors; only the mismatched evaluation half was. FLAW-2 is therefore sharper than “the paper evaluates on the wrong data”: the paper’s own framework, when actually built and run, produces a defensible result that the paper itself never reports, choosing an unrelated dataset instead.

TABLE XXI

On-chain storage growth, daily 1 MiB backups, 4 replicas. Every figure is arithmetic over §IV-D’s measured overhead factor, not a new measurement.

Scenario	Per-node/yr	Cluster/yr ($\times 4$)	Time to 1 TiB/node
Small clinic (50 pts)	20.3 GiB	81.3 GiB	≈ 50.4 yr
Medium hospital (5,000 pts)	1.98 TiB	7.94 TiB	≈ 6.05 mo
Large network (50,000 pts)	19.8 TiB	79.4 TiB	≈ 18.4 days

B. FLAW-4 – evaluation methodology

Section VII resamples BitcoinHeist to 90% ransomware before evaluating, without stating that the resample is bounded by the scarce class: only 41,413 ransomware rows exist in the cited 2,916,697-row dataset, so a 90%-ransomware sample can hold at most $41,413/0.9 = 46,014$ rows – 1.6% of the dataset described two paragraphs earlier. The published 98.98% accuracy is a number from a $\sim 46K$ -row experiment, reported with nothing to mark that it is not the 2.9M-row one. Compounding this, the resulting class balance makes accuracy nearly free: a constant-positive classifier on this split scores 90.0% (§IV-A).

§IV-A and §IV-B measure the consequence directly: reproducing the paper’s own protocol exactly falls 4.56 accuracy points short of published, and an eight-hypothesis ablation across two sessions – the two most plausible leakage sources first, then splitting criterion, resample strategy, single-split variance, feature engineering, and a programmatically-stacked worst case – narrows that bound to 3.58 and then 1.51 points without closing it at any tested cell. We report the residual as unexplained, with a measured and considerably tightened bound, rather than attribute it to a cause we found no evidence for.

C. FLAW-5 – the §V-3 security claim

Section V-3 argues that BSFR-SH resists 51% attacks because it replaces proof-of-work with pBFT. The argument borrows PoW’s 51% threshold and applies it to a consensus mechanism with a different one: pBFT is safe only while fewer than $n/3$ of replicas are Byzantine ($n \geq 3f + 1$). Moving from PoW to pBFT does not raise the fraction of the network an adversary must control – it lowers it, from just over one half to one third. At the paper’s own deployment of four miner nodes, $f = 1$: two colluding replicas (50% of the cluster – above the $f = 1$ bound the cluster actually tolerates, and below the 51% figure the paper cites as the threshold being defended against) can make two honest replicas commit different blocks at the same height. We demonstrate this directly rather than argue it: a test constructs exactly this scenario against our own pBFT implementation and asserts the fork occurs, at exactly the bound f predicts and no tighter.

The fair counter-argument, which §V-3 could have made and does not, is that pBFT’s threshold counts identities, not hash power, and in a permissioned deployment identities cannot be minted by an outside adversary – a property we test separately, showing a swarm of

unauthorised identities cannot form a certificate regardless of size. The defensible version of §V-3’s claim is therefore operational – “an attacker must compromise two of our four named cloud servers” – not consensus-theoretic, and it gets weaker, not stronger, in relative terms as a reason to prefer pBFT over PoW at a fixed and small node count.

D. DEV-26 – preprocessing deletes the positive class

Algorithm 2 line 3 pre-processes honeypot data and, in its entirety, “removes the abnormalities.” The literal reading is statistical outlier removal. On a corpus whose purpose is teaching a detector what ransomware looks like, that reading is self-defeating: ransomware behaviour is the minority class by construction – it is the abnormality – so an outlier-removal step deletes exactly the samples the detector exists to learn from. Applied naively, Algorithm 2’s own pre-processing step would hand Algorithm 3 a clean-looking dataset with the positive class silently gone, and §VII would then report a plausible-looking accuracy computed on data that never saw an attack. We interpret “abnormalities” as sensor defects rather than unusual behaviour (§III-D) and pin the alternative reading’s failure mode with a direct test asserting that cleaning never removes the positive class.

E. DEV-08 – TPS is Amortisation, not Throughput

Section VII-D presents TPS as a measured throughput property that improves as the chain grows across the paper’s three cases. It is not: every published TPS value equals $\text{total_tx}/\text{total_time}$ exactly, and §VII-B measures the direct consequence – once cluster setup is excluded from the timed span, marginal per-block cost and derived TPS are both flat, exactly what “the published rise is amortisation” predicts and what our own implementation lets us test directly rather than infer.

IX. What the Paper Gets Right

Two of the paper’s choices are execution defects, not premise defects, and the distinction matters for how it should be read. The two-chain architecture – one chain for the ground truth a detector trains on, a separate chain for the backups a system restores from, never sharing node sets or state – genuinely does separate detection integrity from recovery integrity: an adversary who compromises the mechanism protecting one does not thereby compromise the other. And the central idea that blockchain immutability can protect a detector’s training data from tampering, in a domain where an attacker has

every incentive to poison exactly that data if they can reach it, is sound, and is not the load-bearing idea of any of the four papers Table II compares against. The paper’s execution of that idea – its evaluation methodology, its security argument for choosing pBFT, its preprocessing specification, and the connection between what it builds and what it measures – has the five gaps above. Both are true, and a critique that lists only the five would misrepresent the paper’s contribution as smaller than it is.

X. Conclusion

We built a complete, from-scratch implementation of BSFR-SH’s five algorithms, both of its independent private blockchains under real pBFT consensus, and both a faithful reproduction of the paper’s published evaluation and the honeypot feature pipeline its own Phase 2/3 requires but never defines. Reproducing the paper’s own methodology exactly, its headline detection result does not reproduce: our best model reaches 0.9442/0.9697 against a published 0.9898/0.990, and an eight-hypothesis ablation across two follow-up sessions narrows, without closing, a gap that tightens from 4.56 to 3.58 to 1.51 accuracy points. We find five further, independent defects – a disconnect between what the framework is designed to detect and what it is evaluated on, a degenerate and undisclosed evaluation split, a security argument that inverts pBFT’s actual threshold relative to proof-of-work, a preprocessing specification that, read literally, deletes the class it exists to detect, and a reported throughput curve that is arithmetic amortisation rather than a measured property. Against this, the paper’s central architectural idea – two chains, separately protecting a detector’s ground truth and a system’s recoverability, from an adversary with every incentive to attack both – is sound, and our own honeypot pipeline shows the framework can produce a real, bounded detection result (0.8422 balanced accuracy) once its undefined layer is actually built.

Twelve extensions, built once that working system existed, tested the architectural idea itself rather than the paper’s published numbers, across three analyses. Security (§V): the four-tier threat model §V never states; formal (Scyther) verification of our own session protocol (Tier 1); a pBFT-vs-Raft comparison finding a single Byzantine leader corrupts Raft’s chain undetectably, which is the reason pBFT, not Raft, is the correct choice here – a reason the paper never states (Tier 2); external anchoring against a simulated public chain, answering §VIII’s own stated future work (Tier 2–3); and a real, measured honeypot data-poisoning attack together with a drift-detection defense built and evaluated against it, closing the loop this project’s own threat model opened. The defense finding is a genuine research contribution, not a critique: it catches the poisoning strategy predicted least catchable (an engineered class-boundary midpoint that, in a balanced corpus, coincides with the population mean)

and misses the one predicted most catchable (label flipping, invisible to a feature-only detector by construction), at a false-positive cost measured rather than assumed. Detection (§VI): the honeypot pipeline’s own end-to-end result; adversarial feature-space evasion finding roughly a quarter of ransomware samples trivially evadable, and a hardening defense that works only at a real, non-trivial accuracy cost and only within a narrow training-budget window, confirmed non-tree-specific by an independent neural-network replication; and transfer evaluation against two real malware datasets (ClAMP, EMBER) finding the honeypot schema’s dynamic behavioural design does not ground in either, because no static dataset can observe what only execution produces. Scalability (§VII): on-chain storage growth from clinical to network scale, and confirmation that the paper’s own reported throughput curve does not scale the way it is presented as scaling. One extension remains unpursued: an asynchronous pBFT variant with modelled network latency beyond the fixed delays §IV-C already sweeps.

Taken together, the three findings point in the same direction rather than opposite ones. A paper whose evaluation methodology cannot be checked because its protocol is underspecified, whose one checkable headline number does not reproduce even under the most literal reading of its own stated split, and whose central architectural idea – once actually built, attacked, and defended – turns out to have real, measured gaps its own security argument has no vocabulary to name, is a paper whose claims should not be relied on as published, at any of the three levels this report checked them. That is a narrower conclusion than “the framework does not work”: our own honeypot pipeline shows a version of Algorithm 3, trained on data Algorithm 2 actually specifies producing, detecting ransomware at a real, bounded, non-degenerate rate, and our own defense against the poisoning attack this project also found shows that at least one of the architecture’s real gaps is a solvable one, measured rather than merely named. The architecture is worth building on. The published numbers, as published, and the informal security argument, as argued, are not evidence that it works or defends as well as claimed.

References

- [1] M. Wazid, A. K. Das, and S. Shetty, “BSFR-SH: Blockchain-Enabled Security Framework Against Ransomware Attacks for Smart Healthcare,” *IEEE Trans. Consumer Electronics*, vol. 69, no. 1, pp. 18–28, Feb. 2023.
- [2] M. Castro and B. Liskov, “Practical Byzantine Fault Tolerance,” in *Proc. 3rd USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 1999.
- [3] C. G. Akcora, Y. Li, Y. R. Gel, and M. Kantarcioglu, “BitcoinHeist: Topological Data Analysis for Ransomware Prediction,” *IJCAI, 2021 (BitcoinHeist Ransomware Address Dataset, UCI Machine Learning Repository; cited in [1] as ref. [15])*.
- [4] A. Almahhadani et al., network-traffic-based ransomware detector evaluated on Locky traffic traces, *IEEE Access*, vol. 7, 2019; cited in [1] as ref. [11].
- [5] J. Hwang et al., dynamic-analysis-based ransomware detector, *Wireless Personal Communications*, vol. 112, 2020; cited in [1] as ref. [12].

- [6] S. Sharmeen et al., semi-supervised ransomware detector evaluated on an independent corpus, *IEEE Access*, vol. 8, 2020; cited in [1] as ref. [13].
- [7] S. I. Bae et al., PE-file-feature-based ransomware detector, *Concurrency and Computation: Practice and Experience*, vol. 32, 2020; cited in [1] as ref. [14].
- [8] T. Pornin, “Deterministic Usage of the Digital Signature Algorithm (DSA) and Elliptic Curve Digital Signature Algorithm (ECDSA),” RFC 6979, IETF, Aug. 2013.
- [9] C. Cremers, “The Scyther Tool: Verification, Falsification, and Analysis of Security Protocols,” in *Computer Aided Verification (CAV)*, LNCS vol. 5123, Springer, 2008, pp. 414–418.
- [10] D. Ongaro and J. Ousterhout, “In Search of an Understandable Consensus Algorithm,” in *Proc. USENIX Annual Technical Conference (ATC)*, 2014, pp. 305–319.
- [11] B. P. Welford, “Note on a Method for Calculating Corrected Sums of Squares and Products,” *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.
- [12] E. S. Page, “Continuous Inspection Schemes,” *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.